



University of Bahrain  
College of Information Technology  
Department of Information Systems  
B.Sc. In Cybersecurity

# **QUERYMIND: INTELLIGENT TEXT-TO-SQL CONVERSION USING RAG-ENHANCED LLMs AND COMPARATIVE MODEL EVALUATION**

**Prepared by**

**Mohamed Almaraghi    202101655**  
**Abdulrahman Bumeajib    202102196**  
**Ali Alsowaidi    202104585**

**For**

**ITCY 499**

**Senior Project**

**Academic Year 2025-2026-Semester 1**

**Project Supervisor:**  
**Dr. Abdulla K. Aldoseri**

**1 December 2025**

# Abstract

Businesses require natural-language access to databases without exposing records to large language models, as this compromises data confidentiality and conflicts with governance mandates. This research paper introduces QueryMind, a system that resolves this issue by transforming natural-language enquiries into secure SQL through a schema-aware retrieval-augmented generation pipeline featuring SELECT-only validation. We tested the system on an e-commerce database with five different model configurations. The best setup, *llama3.1-8b* with *mxbai-embed-large*, achieved 100% accuracy across all benchmark queries. The entire pipeline averaged 33 seconds per query, with 86% of that time dedicated to LLM generation. The smaller models gave answers faster, in an average of 4 to 7 seconds, but they were only 40 to 60 percent accurate. Security tests in five different situations indicated that the system could withstand SQL injection, XSS, and session-hijacking attacks. The performance of local and remote databases was almost the same, with only a 7% difference in total processing times. The results show that bigger LLMs take longer to process, but they make SQL generation more accurate and safer. They also show that a fully local, schema-restricted setup is reliable and keeps data private.

# Acknowledgments

﴿وَقُلْ رَبِّ زِدْنِي عِلْمًا﴾

We begin by expressing our gratitude to God. His support gave us strength, persistence, and patience through every challenge and long working session. His guidance helped us complete this project with confidence.

We thank our supervisor, Dr. Abdulla Aldoseri, for his support and direction. His feedback, time, and clarity shaped our work. His encouragement pushed us to stay focused.

We also thank our families for their support. Their belief in us kept our motivation high throughout this project.

We appreciate our friends for supporting our well-being. Their presence during stressful periods, long study days, and late discussions helped us stay balanced.

Lastly, we acknowledge the effort we invested in this project. The long hours, continuous learning, and commitment played a key role in bringing this work to completion.

# Table of Contents

|                                                      |      |
|------------------------------------------------------|------|
| ABSTRACT .....                                       | II   |
| ACKNOWLEDGMENTS.....                                 | III  |
| LIST OF TABLES .....                                 | VII  |
| LIST OF FIGURES.....                                 | VIII |
| CHAPTER 1 INTRODUCTION .....                         | 1    |
| 1.1 PROBLEM STATEMENT .....                          | 1    |
| 1.2 PROJECT OBJECTIVES .....                         | 2    |
| 1.2.1 <i>Scope</i> .....                             | 2    |
| 1.2.2 <i>Target Audience</i> .....                   | 2    |
| 1.2.3 <i>Impact</i> .....                            | 3    |
| 1.2.4 <i>Limitations</i> .....                       | 3    |
| 1.2.5 <i>Assumptions</i> .....                       | 3    |
| 1.3 CONTRIBUTION.....                                | 3    |
| 1.4 REPORT OUTLINE.....                              | 4    |
| CHAPTER 2 LITERATURE REVIEW .....                    | 5    |
| 2.1 BACKGROUND .....                                 | 5    |
| 2.2 RELATED WORK.....                                | 6    |
| 2.3 SUMMARY .....                                    | 7    |
| 2.4 RELEVANCE TO OUR PROJECT .....                   | 7    |
| CHAPTER 3 PROJECT MANAGEMENT.....                    | 8    |
| 3.1 PROCESS MODEL .....                              | 8    |
| 3.2 RISK MANAGEMENT .....                            | 10   |
| 3.3 PROJECT ACTIVITIES PLAN.....                     | 11   |
| CHAPTER 4 REQUIREMENT COLLECTION AND ANALYSIS.....   | 13   |
| 4.1 REQUIREMENT ELICITATION .....                    | 13   |
| 4.2 SYSTEM REQUIREMENTS.....                         | 14   |
| 4.2.1 <i>Functional Requirements</i> .....           | 14   |
| 4.2.2 <i>Non-Functional Requirements</i> .....       | 15   |
| 4.3 SYSTEM MODELS .....                              | 16   |
| 4.3.1 <i>System Architecture</i> .....               | 16   |
| 4.3.2 <i>Entity Relationship Diagram (ERD)</i> ..... | 17   |

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| CHAPTER 5 SYSTEM DESIGN .....                                                          | 19 |
| 5.1 INTRODUCTION.....                                                                  | 19 |
| 5.2 DATABASE SCHEMA.....                                                               | 19 |
| 5.2.1 Purpose of the Schema in the System .....                                        | 19 |
| 5.2.2 Data Dictionary .....                                                            | 20 |
| 5.2.3 How the System Uses Table Metadata in RAG, Validation, and Query Generation..... | 22 |
| 5.3 DESIGN PRINCIPLES .....                                                            | 22 |
| 5.3.1 Interface 1: Login Page .....                                                    | 22 |
| 5.3.2 Interface 2: Home Page.....                                                      | 23 |
| 5.3.3 Interface 3: Model Setting and Profile .....                                     | 24 |
| 5.3.4 Interface 4: SQL Preview Panel .....                                             | 24 |
| 5.3.5 Interface 5: Results Page.....                                                   | 25 |
| 5.3.6 Navigation and User Flow.....                                                    | 26 |
| 5.4 CORE PROCEDURES AND ALGORITHMS .....                                               | 26 |
| 5.4.1 Schema Extraction and Metadata Loading.....                                      | 27 |
| 5.4.2 RAG Retrieval Process .....                                                      | 27 |
| 5.4.3 Prompt Construction Logic.....                                                   | 28 |
| 5.4.4 SQL Generation Using LLM.....                                                    | 28 |
| 5.4.5 SQL Validation Pipeline .....                                                    | 29 |
| 5.4.6 Read-Only Execution Routine.....                                                 | 29 |
| 5.4.7 Error Handling.....                                                              | 30 |
| 5.5 MAPPING TO REQUIREMENTS.....                                                       | 31 |
| 5.5.1 Functional Requirements Coverage .....                                           | 31 |
| 5.5.2 Non-Functional Requirements Coverage .....                                       | 31 |
| 5.6 SUMMARY .....                                                                      | 32 |
| CHAPTER 6 SYSTEM IMPLEMENTATION AND TESTING .....                                      | 33 |
| 6.1 SYSTEM IMPLEMENTATION .....                                                        | 33 |
| 6.1.1 Development Environment and Hardware Specifications.....                         | 33 |
| 6.1.2 Technology Stack Selection and Justification .....                               | 33 |
| 6.1.3 Project Structure and Module Organization .....                                  | 34 |
| 6.2 EVALUATION AND TESTING .....                                                       | 34 |
| 6.2.1 Testing Methodology.....                                                         | 35 |
| 6.2.2 Security Testing .....                                                           | 36 |
| 6.2.3 Performance Measurements.....                                                    | 38 |
| 6.2.4 Model Comparison Experiments .....                                               | 40 |
| 6.2.5 Local vs Remote Database Performance.....                                        | 41 |
| 6.2.5.1 Test Configuration .....                                                       | 42 |

|                                                      |    |
|------------------------------------------------------|----|
| 6.2.5.2 Performance Comparison.....                  | 42 |
| 6.2.6 Comparison with Real Solution .....            | 44 |
| 6.2.6.1 Analysis.....                                | 46 |
| 6.2.7 Strength and weaknesses Analysis.....          | 46 |
| 6.2.8 Discussion and Comparison with Literature..... | 47 |
| CHAPTER 7 CONCLUSION AND FUTURE WORK.....            | 49 |
| REFERENCES .....                                     | 50 |
| APPENDIX A.....                                      | 51 |
| PROJECT WEBSITE & RESOURCES .....                    | 52 |

## List of Tables

|                                                        |    |
|--------------------------------------------------------|----|
| Table 1. Identified Risks for the Text-to-SQL.....     | 10 |
| Table 2. Project Activity Plan .....                   | 11 |
| Table 3. Elicited Needs of Project.....                | 14 |
| Table 4. Customers Metadata .....                      | 20 |
| Table 5. Employees Metadata .....                      | 20 |
| Table 6. Offices Metadata .....                        | 20 |
| Table 7. Orders Metadata .....                         | 21 |
| Table 8. Orderdetails Metadata .....                   | 21 |
| Table 9. Payments Metadata.....                        | 21 |
| Table 10. Productlines Metadata .....                  | 21 |
| Table 11. Products Metadata .....                      | 21 |
| Table 12. Hardware Configuration.....                  | 33 |
| Table 13. Technology Stack .....                       | 34 |
| Table 14. Test Dataset .....                           | 36 |
| Table 15. Security Testing Results .....               | 36 |
| Table 16. Core Function Performance Metrics.....       | 39 |
| Table 17. Model Combination Result.....                | 40 |
| Table 18. Query Complexity Analysis .....              | 41 |
| Table 19. Local vs Remote Performance Comparison ..... | 42 |
| Table 20. QueryMind vs text2sql.ai Comparison .....    | 44 |
| Table 21. Feature Comparison Matrix.....               | 45 |
| Table 22. System Strength.....                         | 46 |
| Table 23. System Weaknesses.....                       | 47 |

## List of Figures

|                                                        |    |
|--------------------------------------------------------|----|
| Figure 1. Phases of the Waterfall SDLC .....           | 9  |
| Figure 2. Project Gantt Chart.....                     | 12 |
| Figure 3. System Architecture.....                     | 16 |
| Figure 4. The Entity Relationship Diagram (ERD) .....  | 18 |
| Figure 5. Login Page .....                             | 23 |
| Figure 6. Home Page .....                              | 23 |
| Figure 7. Table Metadata.....                          | 24 |
| Figure 8. Navigation Bar .....                         | 24 |
| Figure 9. SQL Preview .....                            | 25 |
| Figure 10. Result Page.....                            | 26 |
| Figure 11. User Flow .....                             | 26 |
| Figure 12. Project Structure.....                      | 34 |
| Figure 13. Gold Dataset Example.....                   | 35 |
| Figure 14. Results Example.....                        | 35 |
| Figure 15. SEC-01 .....                                | 37 |
| Figure 16. SEC-02 .....                                | 37 |
| Figure 17. SEC-03 .....                                | 37 |
| Figure 18. SEC-04.....                                 | 38 |
| Figure 19. SEC-05 .....                                | 38 |
| Figure 20. Time Distribution Pie Chart.....            | 39 |
| Figure 21. Model Accuracy .....                        | 40 |
| Figure 22. Average SQL Generation Time.....            | 41 |
| Figure 23. Local DB via Adminer .....                  | 42 |
| Figure 24. Remote DB Credentials.....                  | 42 |
| Figure 25. Remote DB via Adminer.....                  | 42 |
| Figure 26. Local vs Remote Total Generation Time ..... | 43 |
| Figure 27. Time Component Breakdown .....              | 43 |
| Figure 28. Accuracy Comparison.....                    | 45 |



# Chapter 1

## Introduction

This chapter introduces the topic of text-to-SQL systems supported by RAG and outlines the motivation behind developing a reliable, schema-aware approach. Section 1.1 presents the problem statement and the gaps observed in current text-to-SQL methods that form the basis of this project. Section 1.2 describes the objectives of the work, including the system's scope and the constraints followed during development. Section 1.3 highlights the relevance of the project and the value it provides for improving accuracy, safety, and usability in database querying.

### 1.1 Problem Statement

Relational databases are widely used by organizations to store and maintain transactional, financial and customer data. Making anything useful of that data usually involves writing longish SQL queries, skills which are generally possessed by database administrators or developers. This technical limitation alienates much of the decision-making audience, analysts and non-technical users who perform their own data exploration or analysis.

So, the common issue is that it's not easy to convert natural-language questions to SQL. Manual clause generation is time-consuming, error-prone and unmanageable across business units. Plus, exposing (without any kind of restriction) whole databases to the user interfaces may cause quite a bit of trouble, if queries are not properly validated or run within some limiting context.

Text-to-SQL systems powered by large language models (LLMs) offer a promising solution. However, hallucinations; where the model fabricates table names or columns remain a serious risk. Retrieval-Augmented Generation (RAG) mitigates this by constraining the model's knowledge to verify schema components, making the query generation more accurate and secure.

The importance of this capability is growing. A report found that companies leveraging data-driven decision-making are 23 times more likely to acquire customers, six times more likely to retain them, and 19 times more likely to be profitable [1]. Yet, another paper notes that only 20% of decision-makers have direct access to the data they need, citing technical complexity as a major blocker [2].

Organizations increasingly use relational databases to store and manage data. Our system fills this gap by allowing users to query data using arbitrary natural-language questions, all while enforcing schema awareness, validation, and read-only execution. This makes it especially valuable in spaces where security and auditable truth are paramount, like in finance, healthcare, and government.

## **1.2 Project Objectives**

This project aims to develop a secure and schema-aware, query-based Text-to-SQL system which can be used by the users to retrieve records from the database using natural language-based questions. The system leverages models such as Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) to efficiently convert natural language expressions made by a user into sound and validated SQL SELECT queries that respect data security.

### **1.2.1 Scope**

The project will comprise the development of a prototype that receives natural-language input, retrieves relevant schema context through semantic search, formulates an accurate LLM prompt, and executes a SQL SELECT query in possibly a sandbox. The prototype consists of a user interface, retrieval engine, LLM Processing layer, and validator. It is designed to work with structured relational databases, specifically MySQL, and operates read-only using SELECT queries. Evaluation is performed with a controlled dataset and using predefined user questions, where we measure the accuracy, latency and LLM robustness across various LLM and embedding combinations.

### **1.2.2 Target Audience**

Analysts, auditors, business users, and technical personnel who need to get data without manually writing SQL are among the system's target users. Additionally, academic

institutions interested in applied natural language processing and AI for structured data access might use it as research and learning tool.

### **1.2.3 Impact**

This project is designed out of students in the University of Bahrain, but these are universal specifications and can be implemented all around the world. This adds to the flexibility of the system for wider deployment in other industries and regions. The focus is educational and exploratory with an eye to future extensions that could support industrial scale deployments or the integration in existing business intelligence systems.

### **1.2.4 Limitations**

This version of the system does not support any form of data manipulation such as INSERT, UPDATE, or DELETE commands. It only accepts English-language questions and operates on one database schema at a time. It is also limited to MariaDB and MySQL relational databases and does not support other types of database or NoSQL data sources.

### **1.2.5 Assumptions**

The system presumes that users access via legitimate database credentials and function within a secure and verified environment. It expects the language model to only get schema context, not live data or sensitive values. The design also assumes that ChromaDB, the vector store, is installed, set up, and isolated for each session so that schema embeddings stay linked to the right user. The backend further assumes that database permissions enforce read-only access, and that the host environment blocks unauthorized files, networks, or model-level actions. These assumptions let the system focus on making SQL in a controlled way and running queries safely within certain limits.

## **1.3 Contribution**

1. This project presents a schema-aware Retrieval-Augmented Generation method that confines the LLM to the authentic database schema, effectively mitigating hallucinations in all evaluated queries and allowing the optimal model configuration to achieve complete accuracy.
2. We use a strict validation pipeline that makes sure SQL generation is safe and only allows reading. The system blocked all non-SELECT attempts, such as UPDATE, DELETE, and DROP, in five security tests, resulting in a 100 percent pass rate.

3. We test five LLM and embedding model combinations on a fixed set of five queries to get consistent metrics for SQL accuracy (40-100 percent), latency (4-33 seconds), and failure modes that depend on model size and query complexity.
4. The system runs completely on its own using open-source models, unlike black-box cloud APIs. The whole pipeline took an average of 33 seconds per query, with 86% of that time spent on LLM generation. No data ever left the device.
5. The prototype that came out of this is easy to audit, can be expanded, and is good for industries that care about privacy. Its performance stayed the same whether it was running on a local or remote server, with only a 7% difference in total processing times.

## **1.4 Report Outline**

The remainder of this report is organized as follows. Chapter 2 presents a literature review covering key research and technologies related to Text-to-SQL systems, large language models, and retrieval-augmented generation. Chapter 3 outlines the project management approach, including timelines, milestones, and team roles. Chapter 4 details the process of requirement collection and analysis, identifying functional, non-functional, and technical needs of the system. Chapter 5 describes system design, focusing on architectural components, data flow, and security considerations. Chapter 6 discusses the implementation and testing of the system, including environment setup, code structure, and evaluation results. Finally, Chapter 7 summarizes the main findings of the project and proposes areas for future improvement and research.

## Chapter 2

# Literature Review

This chapter reviews the foundations of natural-language access to databases and summarizes the most relevant systems and studies that influence the design of QueryMind. It has two main sections. Section 2.1 presents background concepts, including the evolution of Text-to-SQL, schema-aware approaches, and recent advances in LLMs and Retrieval-Augmented Generation (RAG). Section 2.2 presents related work, including commercial and academic systems. The chapter concludes with a summary of key findings in Section 2.3 and a discussion of how these findings relate to our project in Section 2.4.

### 2.1 Background

Early studies on natural language interfaces to databases concentrated on rule-based or template-based approaches that depended on fixed grammar and manually crafted patterns. These systems could handle simple questions, but they couldn't handle complex schemas or queries that involved more than one table. Neural sequence-to-sequence models made things more flexible by mapping natural language inputs directly into SQL statements, with the help of benchmark datasets like Spider. But these models had a hard time with domain shift, schema complexity, and making SQL that was unsafe or imagined.

Recent advancements in large language models (LLMs) have improved natural-language understanding and structured output generation, enabling more accurate Text-to-SQL systems. Modern LLM deployment tools, such as Ollama, allow organizations to run models locally without relying on external APIs. This supports privacy-sensitive use cases where database records cannot be exposed to cloud services. Open-source LLMs, including Llama, Gemma, and Qwen variants, provide controllable inference environments while maintaining competitive performance.

Retrieval-Augmented Generation (RAG) has emerged as a key technique for improving accuracy and security in Text-to-SQL. Instead of giving the model full access to database content, RAG provides only schema metadata or the specific tables relevant to a user query.

This guided retrieval reduces hallucinations and constrains the model to valid table and column names. Industry sources highlight that restricting LLM access to metadata is essential for governance, privacy, and controlled SQL generation [3]. Academic work such as Zhai et al. shows that execution feedback and structured reasoning can further improve accuracy and reduce invalid queries [4].

Together, these developments establish the foundation for modern schema-aware, secure, and locally deployed Text-to-SQL systems like QueryMind.

## 2.2 Related Work

This section presents the systems and research-related work papers. It includes real-world tools, academic frameworks, and enterprise deployments relevant to the design of QueryMind.

1. Exasol Governed Text-to-SQL System

Exasol proposes an on-premises Text-to-SQL framework built on strict governance policies. The system enforces read-only SQL, restricts LLM visibility to metadata, and prevents access to raw records [3]. It emphasizes operational control but lacks published accuracy or latency benchmarks.

2. Zhai et al. (2025) Execution-Feedback Framework

Zhai et al. introduce ExCoT-DPO, an execution-driven optimization method that improves Text-to-SQL accuracy using chain-of-thought reasoning and reinforcement signals based on execution correctness [4]. While experimentally strong, the work does not address governance, real-world deployment, or security concerns.

3. LinkedIn Production Text-to-SQL System

Chen et al. describe LinkedIn’s enterprise text-to-SQL platform powered by metadata graphs, query logs, and schema-aware retrieval [5]. The system enables large-scale internal analytics but achieves around 53 percent accuracy and provides limited transparency into its security and latency characteristics.

4. text2sql.ai Commercial Platform

text2sql.ai is a cloud-based Text-to-SQL service supporting more than twelve SQL engines, schema uploads, and an interactive interface [6]. Strengths include multi-dialect support and ease of use. Limitations include subscription requirements, reliance on third-party infrastructure, and reduced privacy for sensitive workloads.

## 2.3 Summary

The reviewed literature leads to several important observations:

- LLM-based Text-to-SQL is increasingly viable, especially when paired with schema-aware retrieval.
- Privacy, governance, and auditability remain major concerns for organizations handling sensitive data.
- Metadata quality strongly affects accuracy, especially for multi-table queries or large schemas.
- Benchmarks show promising accuracy, yet real-world deployment is limited by token constraints, latency, and safety concerns.
- Cloud-based tools deliver convenience but sacrifice control and confidentiality, motivating interest in local or open-source solutions.

## 2.4 Relevance to Our Project

Given prior findings in the literature, our study addresses three knowledge gaps particularly well. First, in the Exasol system, read-only safe SQL generation is enforced by restricting the LLM to metadata and running every query inside secure process [3]. Second, Zhai et al. (2025) point out how important it is to assess and tune LLM output based on the execution feedback, justifying our choice of real databases as well for comparing different LLM and embedding model pairs [4]. Third, the business model proposed by Chen et al. (2025) in LinkedIn, also demonstrates that the performance and governance benefits of retrieval layers using schema metadata and historic frequency successful queries. We learn from these works extensive best practices (including metadata incorporation, prompt engineering, query verification and benchmarking) but extend them to a framework of more principled and reproducible evaluation where the focus is on data governance and real-world applicability [5]. The comparison with text2sql.ai shows how commercial tools rely on cloud infrastructure and broad database support to provide accuracy and convenience, although this comes with cost and reduced control. Our system offers a free and fully local alternative, suitable for privacy-sensitive environments, and provides a foundation for extending language support and safe write-operations in future versions [6].

## **Chapter 3**

# **Project Management**

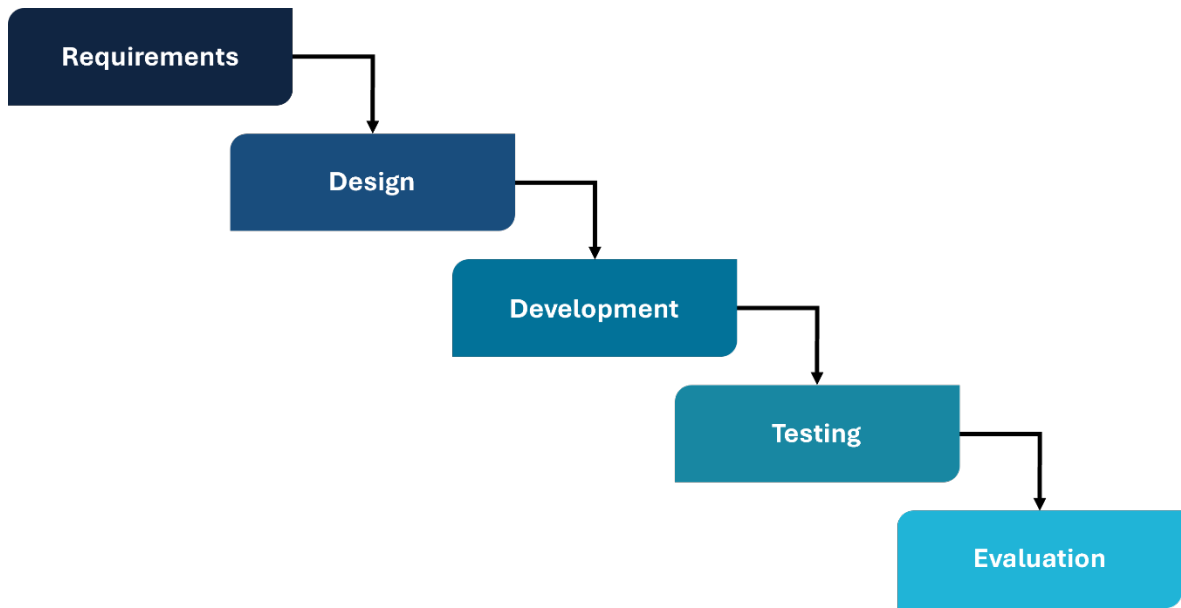
This chapter explains the management approach used to plan, structure, and execute QueryMind. Section 3.1 describes the chosen Software Development Life Cycle (SDLC). Section 3.2 presents the risk management process applied throughout development. Section 3.3 outlines the project activity plan, including task sequencing, timelines, and dependencies. Together, these sections show how the system was organized from initial requirements to implementation, testing, and evaluation.

### **3.1 Process Model**

The Waterfall Software Development Life Cycle was chosen because this project went from requirements to evaluation in one full pass, without any iterative sprints. The Waterfall model is a linear and structured way of doing things. You must finish one phase before you can move on to the next. It is appropriate for scholarly and technical endeavors characterized by well-defined specifications and restricted iteration cycles [7]. Waterfall also works well in situations where the system's scope and limits are known ahead of time, like a controlled Text-to-SQL environment with fixed database schemas, set security rules, and set evaluation metrics.

Waterfall supports predictable planning, clear documentation, and sequential verification steps, which are essential when dealing with components like schema extraction, SQL validation, and LLM pipeline integration. Since LLM behavior depends heavily on stable prompts, schema context, and validation rules, following a linear design–build–test structure reduces uncontrolled variability and supports reproducible results.





*Figure 1. Phases of the Waterfall SDLC*

*Figure 1. Phases of the Waterfall SDLC* shows the phases of the Waterfall SDLC. The following describes how each phase was applied in this project:

- **Requirements phase:** Defined project goals, including enforcing SELECT-only SQL generation, identifying schema-aware retrieval needs, and mapping functional and non-functional requirements.
- **Design phase:** Produced the full system architecture, including the schema loader, RAG retriever, LLM module, SQL validator, and execution engine.
- **Implementation phase:** Developed the modules using Python, Flask, ChromaDB, and Ollama, integrating each component sequentially according to the design.
- **Testing phase:** Used a fixed question dataset to evaluate SQL accuracy, result correctness, and performance across five model configurations.
- **Evaluation phase:** Compared QueryMind with a real-world Text-to-SQL platform and analyzed performance, accuracy, and security outcomes.

Using Waterfall allowed steady progression through clearly defined stages and ensured that each component met structural, functional, and security requirements before moving to the next phase.

## 3.2 Risk Management

Risk management is very important for projects that use cutting-edge technologies like LLMs, vector search, and dynamic schema retrieval. The Waterfall model stresses finding risks early, fixing them before implementation, and stopping problems from spreading into later phases [8]. Structured risk identification was necessary because LLMs are unpredictable and SQL validation needs to be safe.

There were risks in this project, such as schema extraction failures, wrong RAG retrieval, SQL injection attempts, and LLM hallucinations. Good documentation helped keep the team on the same page and made sure that security and reliability were always top priorities during development. *Table 1. Identified Risks for the Text-to-SQL* shows all the risks that have been found, how serious they are, and the steps that have been taken to reduce them.

*Table 1. Identified Risks for the Text-to-SQL*

| Risk Description                                                | Level  | Mitigation Strategy                                                                                              |
|-----------------------------------------------------------------|--------|------------------------------------------------------------------------------------------------------------------|
| 1. Incorrect schema extraction during login                     | Medium | Validate table list and DDL output before embedding. Re-run extraction on failure.                               |
| 2. RAG retrieves irrelevant tables                              | Medium | Use stricter similarity thresholds and adjust top-k retrieval. Monitor mismatches during testing.                |
| 3. LLM generates SQL using non-existent table or column names   | High   | Enforce strict schema validation before execution and return a controlled error.                                 |
| 4. Slow inference time for large LLMs                           | Medium | Allow users to choose smaller models for quick tests. Add hardware acceleration when available.                  |
| 5. Vector store corruption or misalignment across sessions      | Medium | Create isolated Chroma directories per session and clear them on logout.                                         |
| 6. SQL injection attempts through natural-language input        | High   | Apply SQL extraction and SELECT-only enforcement. Block any non-SELECT patterns.                                 |
| 7. Database credential misuse                                   | High   | Secure session storage, short session lifetime, and no server-side persistence of credentials.                   |
| 8. Schema changes in the database during an active session      | Low    | Implement load schema only to read tables and columns, using RAG, chunk those tables and then pass it to the LLM |
| 9. External solution benchmarking produces incompatible results | Low    | Normalize differences across SQL dialects and limit evaluations to logical equivalence.                          |

### 3.3 Project Activities Plan

The project plan follows Waterfall SDLC's step-by-step approach. Before the next phase starts, the previous one must be finished. Also, no implementation work can start until the full design is done. This structure makes it possible to plan, keep good records, and test things in a controlled way, which are all good for developing systems based on research. The table below shows the full activity plan that was used to make QueryMind.

Table 2. Project Activity Plan

| Phase          | Task Description                                                                           | Start Date  | End Date    |
|----------------|--------------------------------------------------------------------------------------------|-------------|-------------|
| Requirements   | Defined project goal: convert natural-language queries into safe SELECT-only SQL using RAG | 01-Sep-2025 | 05-Sep-2025 |
|                | Mapped functional requirements, user stories, and security constraints                     | 03-Sep-2025 | 08-Sep-2025 |
|                | Identified risks and constraints, decided on RAG + LLM architecture                        | 05-Sep-2025 | 10-Sep-2025 |
| Design         | Designed modular architecture (schema loader, retriever, LLM engine, validator, executor)  | 08-Sep-2025 | 15-Sep-2025 |
|                | Planned database structure, dummy data creation, and indexing layout                       | 12-Sep-2025 | 18-Sep-2025 |
|                | Defined prompt strategy, SQL-extraction logic, validation flow                             | 14-Sep-2025 | 20-Sep-2025 |
| Implementation | Built database config, schema loader, and SQL execution module                             | 18-Sep-2025 | 24-Sep-2025 |
|                | Implemented LLM engine with Ollama models and prompt control                               | 24-Sep-2025 | 05-Oct-2025 |
|                | Built Flask frontend (input, SQL display, results page)                                    | 01-Oct-2025 | 10-Oct-2025 |
|                | Added SQL sanitizer, SELECT-only enforcement, identifier validation                        | 05-Oct-2025 | 15-Oct-2025 |
| Testing        | Prepared gold-questions dataset and expected outputs                                       | 12-Oct-2025 | 18-Oct-2025 |
|                | Ran experiments with multiple embedding models and LLMs                                    | 18-Oct-2025 | 30-Oct-2025 |
|                | Debugged invalid SQL, identifier issues, and extraction failures                           | 20-Oct-2025 | 02-Nov-2025 |
|                | Collected metrics: SQL accuracy, result match, latency                                     | 25-Oct-2025 | 05-Nov-2025 |
| Evaluation     | Compared results against a real-world solution                                             | 05-Nov-2025 | 15-Nov-2025 |
|                | Analysis, and final documentation                                                          | 12-Nov-2025 | 27-Nov-2025 |

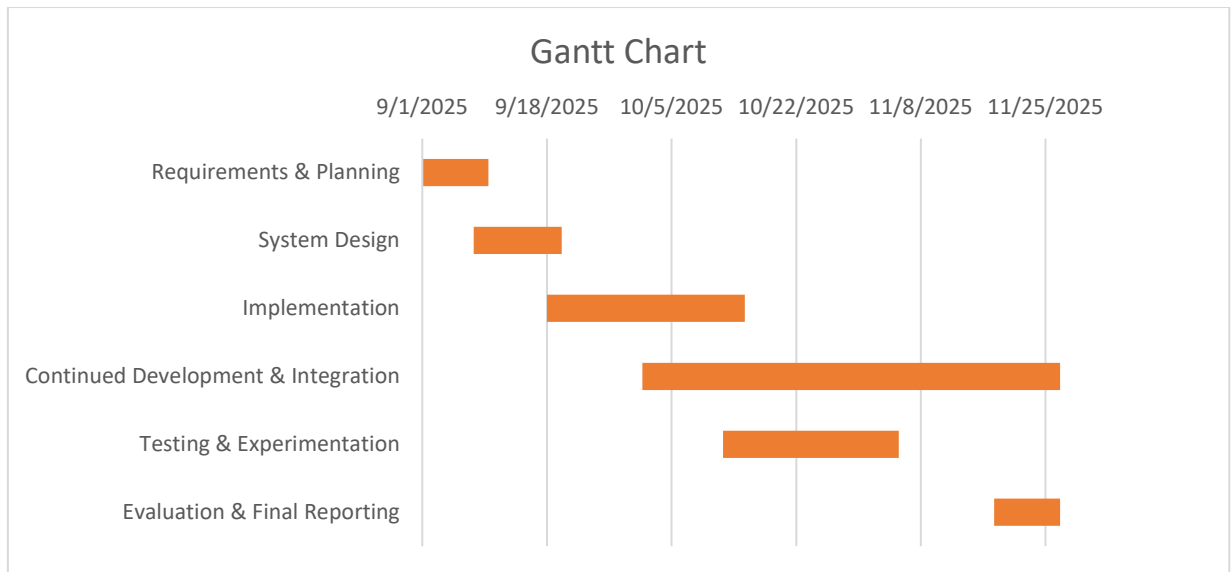


Figure 2. Project Gantt Chart

The Gantt chart outlines the full timeline of the project according to Waterfall SDLC. It shows how each phase progressed in sequence, starting with requirements and planning, followed by system design and implementation. Continued development and integration occurred after the implementation phase, leading into structured testing and experimentation. The final reporting and evaluation phase concludes the schedule. This visualization highlights the linear flow of tasks and the dependency structure typical of the Waterfall model, where each phase must be completed before the next begins [7], [8].

## **Chapter 4**

# **Requirement Collection and Analysis**

Chapter 4 discusses how the system requirements for QueryMind were found, checked, and turned into measurable specifications. The chapter connects user's needs to specific system functions and shows how each requirement fits into the Text-to-SQL pipeline. This includes schema extraction, RAG retrieval, prompt design, SQL validation, execution, and logging. It also sets the standards for success that will be used in the testing and evaluation phases, like accuracy, latency, security, and auditability.

Section 4.1 talks about how interviews, observations, and feedback on early prototypes were used to get requirements. Part 4.2 shows the functional and non-functional requirements that came out of this process. Section 4.3 shows system models like the architecture diagram, Entity-Relationship Diagram, and flow representations that will help with the design in later chapters.

### **4.1 Requirement Elicitation**

We got the requirements by doing semi-structured interviews, short observation sessions, and looking at logs from earlier prototype experiments. This combination made sure that the final scope was based on real user needs and real-world limits. The interviews helped confirm what people thought about safety, accuracy, usability, and how to handle failure. Watching a live demo showed how users asked questions, understood system messages, and used the interface. Log analysis gave us measurable starting points for latency, SQL correctness, and the kinds of errors.

Three people gave feedback: two from Gulf Petrochemical Industries Company (GPIC) and one from the Arab International Cybersecurity Conference & Exhibition. Their backgrounds in managing cybersecurity, developing software, and analyzing cybersecurity made sure that they had a balanced view of what needed to be done. There were four main themes that came up in all the sessions: strict read-only operation, schema-only exposure for the LLM, clear and safe error messages, and measurable accuracy and timing expectations.

Table 3. Elicited Needs of Project

| Job Title                          | Goals and Pain Points                                      | Elicited Requirements                                                        | Evidence                           |
|------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------|
| Cybersecurity Administrator (GPIC) | Prevent data leakage and enforce controlled access         | SELECT-only execution, table/column whitelisting, block unknown identifiers  | Interview notes, log review        |
| Software Developer (GPIC)          | Stable user experience and fast failure feedback           | Clear validation errors, explainable SQL output, efficient schema retrieval  | Demo observations, prototype tests |
| Cybersecurity Analyst (AICS)       | Trustworthy system behavior and safe NL-to-SQL translation | Transparent SQL previews, understandable results, schema-only model exposure | Conference interview notes         |

## 4.2 System Requirements

This section describes the expected behavior and features of the Text-to-SQL system. It includes both functional and non-functional requirements. Functional requirements define the core actions the system must perform. Non-functional requirements specify the qualities the system must meet during real use, such as security, speed, usability, and maintainability.

### 4.2.1 Functional Requirements

These requirements describe the core functions that enable the Text-to-SQL system to operate as intended. Each requirement can be validated by running the full pipeline and confirming the generated SQL, returned results, and system logs match the expected behavior.

- **Natural-Language Query Input**
  - Type your questions in plain English into one box. Next, it shows the SQL that was made, the time it took to run, and the result rows. Users don't need to know SQL.
- **Schema-Aware Query Generation (LLM + RAG)**
  - Using Retrieval-Augmented Generation, the system only gets the database schema that is needed and sends it to the LLM. The LLM uses this context to make SQL that fits the structure of the database without ever seeing real data.
- **Read-Only SQL Enforcement**
  - The system can only make SELECT statements in SQL. You can't run or create UPDATE, DELETE, INSERT, or DROP queries. This keeps the data safe and follows the least privileged principle.

- **Model Benchmarking and Comparative Evaluation**
  - The system lets you test different combinations of LLM and embedding models against a set of questions that don't change. It automatically sends reports on SQL accuracy, result correctness, latency, and failure patterns to help with repeatable evaluation.
- **Dynamic Schema Visibility Based on User Role**
  - Users who are logged in can only see the tables and columns they have permission to see. Schema retrieval, RAG context, and LLM behavior change based on user permissions to protect sensitive structures.
- **Structured Error Handling**
  - When queries fail validation, when schema context is missing, or when model outputs don't follow the expected formats, the system sends clear and useful messages. It also deals with timeouts and results that are empty.

#### 4.2.2 Non-Functional Requirements

The non-functional requirements define the qualities the system must maintain across all features to ensure security, reliability, and usability.

- **Security** is a primary focus. Database credentials operate under least-privilege access, and the SQL validator blocks any operation that is not a SELECT statement. The LLM never receives live records and only processes schema text, which prevents unintended data exposure.
- **Confidentiality** is maintained through schema-only retrieval. The RAG module provides the model with metadata rather than actual table contents, and user permissions ensure that only authorized structures are visible. Error messages are intentionally generic to avoid leaking sensitive details.
- **Performance and scalability** are supported by schema chunking and vector storage, allowing the system to handle large schemas efficiently. Retrieval focuses on relevant tables only, reducing context size and improving response times. Model selection also enables quick demonstrations and controlled experiments.
- **Usability** is strengthened by a simple interface with a single input box, a table viewer, example queries, and clear timing indicators. Results are displayed on a clean, scrollable table that is easy to interpret.

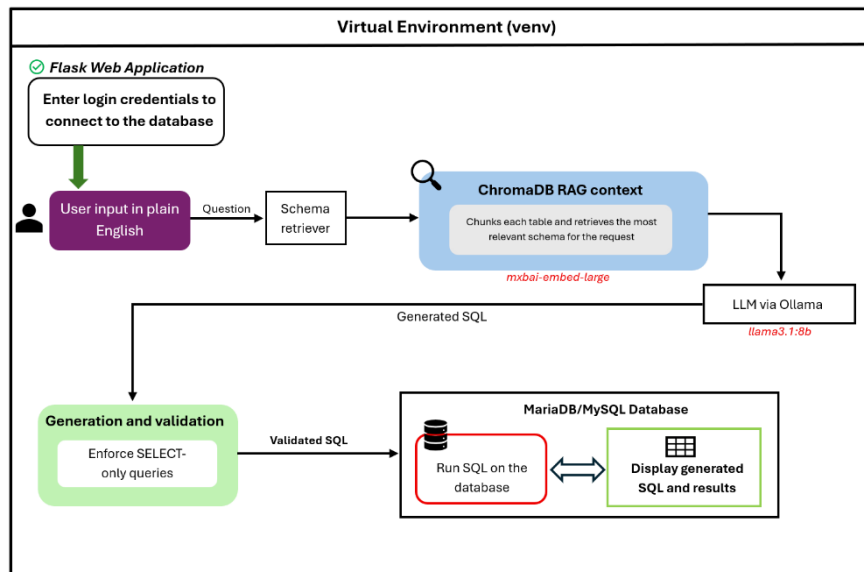
- **Maintainability** is ensured through modular design. Each component: schema loader, retriever, LLM engine, validator, and executor is independent, making updates and replacements straightforward. All experiments and settings are organized systematically to support reproducibility and future enhancements.

## 4.3 System Models

The system model in this section describes how QueryMind operates internally and how its components work together to convert natural-language input into safe SQL output. These models illustrate the system's architecture, data flow, and structural relationships, providing a technical foundation for the design and implementation described in later chapters. Section 4.3.1 explains the end-to-end architecture, showing each processing stage from user input to SQL execution. Section 4.3.2 presents the Entity Relationship Diagram (ERD) of the database used for testing and benchmarking, highlighting how schema structure influences retrieval, validation, and query generation. Together, these models offer a complete view of how the system processes requests and maintains security, accuracy, and schema consistency.

### 4.3.1 System Architecture

*Figure 3. System Architecture* shows the architecture of the Text-to-SQL system. Users interact through the Flask web application, submit a natural-language question, and receive a tabular result along with the generated SQL. The request moves through schema retrieval, RAG context construction, LLM generation, SQL validation, and controlled read-only execution. Only schema metadata is shared with the model. No records leave the database process.



*Figure 3. System Architecture*



The system architecture has a structured pipeline that changes natural-language questions into safe SQL queries through a series of tightly controlled steps. The user starts interacting with the system through the Flask web interface, where they log in (connect to the database) and ask a question. They can only see the tables that their role allows. From the start, this access control follows the least privilege rules. The schema loader obtains table names, columns, and relationships from the connected database when a question is sent. The vector index contains these metadata fragments, which the retriever uses to determine the schema elements that are most relevant to the user's question. This retrieval step makes sure that the LLM acquires a compact and accurate schema context that only has structural information and never actual records. This keeps things private while also helping the model create SQL that fits the schema.

After the context is retrieved, the chosen schema segments and the user's question are sent to the LLM, which makes an SQL query that fits the structure of the database. Thereafter, the output goes to the validation layer, where strict safety rules are used to check the SQL. The validator checks to make sure that the statement is a SELECT query, that all the tables and columns it refers to exist, and that there are no harmful patterns like UPDATE, DELETE, INSERT, or DROP. Only queries that pass these checks are run. The executor runs the SQL with read-only credentials so that data can't be changed by mistake, and the results are sent back to the user in a clear table format. The system presents a clear and safe error message if something goes wrong at any point, such as invalid identifiers, unsafe SQL, or missing schema context. This design makes sure that the whole Text-to-SQL workflow is accurate, has a consistent schema, and runs safely.

#### 4.3.2 Entity Relationship Diagram (ERD)

*Figure 4. The Entity Relationship Diagram (ERD) shows the Entity Relationship Diagram of the e-commerce database used throughout development, testing, and benchmarking. This schema was adopted from the official MySQL sample database provided by MySQLTutorial.org [9]. It includes eight relational tables: *productlines*, *products*, *customers*, *employees*, *offices*, *orders*, *orderdetails*, and *payments*.*

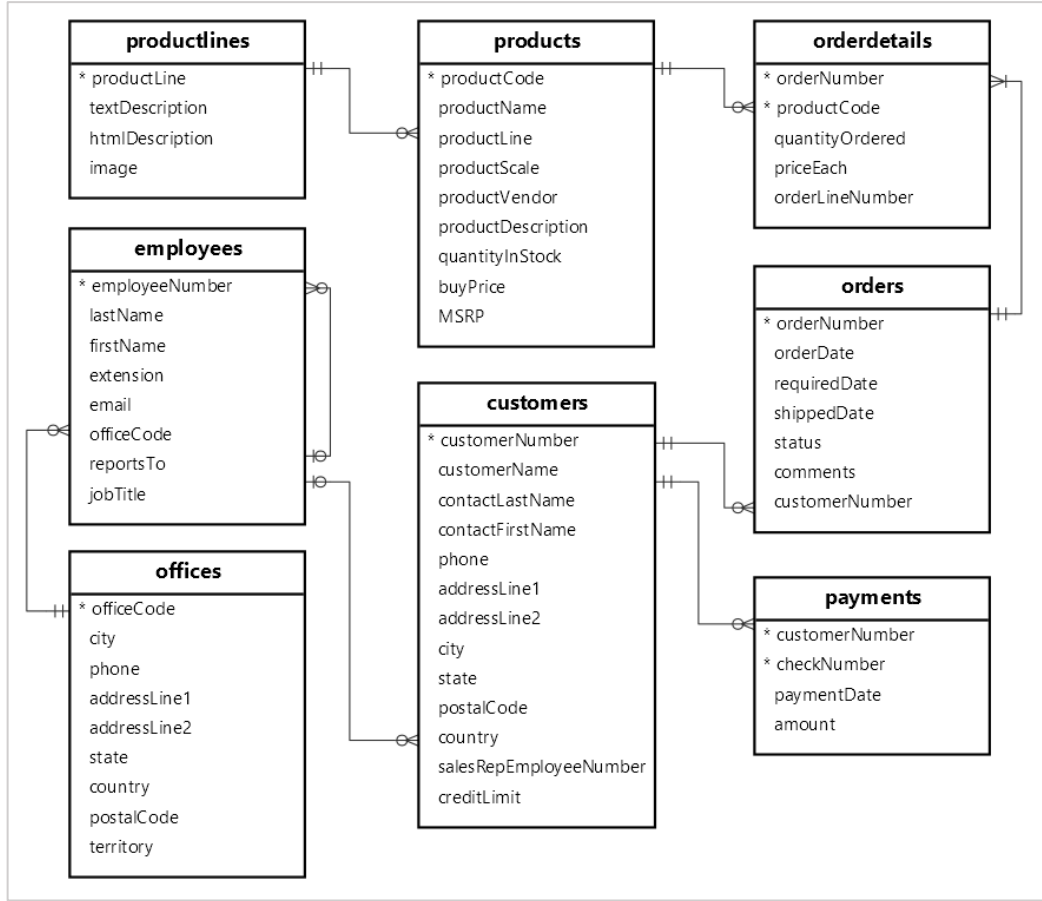


Figure 4. The Entity Relationship Diagram (ERD)

The ERD represents the full structure of the e-commerce database used in this project and serves as the foundation for QueryMind’s schema-aware Text-to-SQL pipeline. The relationships between tables, such as customers linked to orders, products linked to order details, and employees associated with offices, provide a realistic environment for generating and validating multi-table SQL queries. These connections are essential for testing JOIN operations, aggregations, and filters, allowing the system to evaluate how well the LLM handles real relational constraints. Because architecture relies on schema-only retrieval, the ERD directly influences the RAG process, validation logic, and SQL generation accuracy. It ensures that the system interacts with a structured, relational dataset that reflects practical business scenarios, making it suitable for benchmarking both performance and correctness in controlled local and remote deployments.

# Chapter 5

## System Design

### 5.1 Introduction

This chapter gives a full picture of the QueryMind system design, including the architectural choices, database schema, user interface layout, and core logic that make safe natural-language-to-SQL translation possible. The system was built to fix the problems that were found in [Chapter 1](#): making it easier to query databases, lowering the risk of running SQL incorrectly or in an unsafe way, and allowing for flexible model selection for accuracy and performance benchmarking. There is a lot of information about important parts like schema-aware retrieval, LLM prompt construction, SELECT-only validation, and result handling.

The design choices made here are based on the functional and non-functional requirements listed in [Chapter 4.2](#). This is especially true for role-based access, error feedback, and modularity for testing different model combinations. The system's architecture allows for database connections both locally and remotely, and it can handle larger schemas. This chapter connects the requirements and limitations talked about earlier with the actual implementation and evaluation that will be covered in [Chapter 6](#).

### 5.2 Database Schema

#### 5.2.1 Purpose of the Schema in the System

The database schema is a structural map of the database that shows the tables, columns, data types, and relationships. The schema is very important for QueryMind because it helps the system understand how the database is set up and correctly turn natural language questions into valid SQL queries. The schema helps the system make sure that the queries it makes use the right table and column names, keep referential integrity, and follow the rules of the database.

## 5.2.2 Data Dictionary

Table 4. Customers Metadata

| Customers              |               |                                |
|------------------------|---------------|--------------------------------|
| Component              | Type          | Notes                          |
| customerNumber         | int(11)       | PK                             |
| customerName           | varchar(50)   |                                |
| contactLastName        | varchar(50)   |                                |
| contactFirstName       | varchar(50)   |                                |
| phone                  | varchar(50)   |                                |
| addressLine1           | varchar(50)   |                                |
| addressLine2           | varchar(50)   | nullable                       |
| city                   | varchar(50)   |                                |
| state                  | varchar(50)   | nullable                       |
| postalCode             | varchar(50)   | nullable                       |
| country                | varchar(50)   |                                |
| salesRepEmployeeNumber | int(11)       | FK → employees(employeeNumber) |
| creditLimit            | decimal(10,2) | nullable                       |

Table 5. Employees Metadata

| Employees      |              |                                |
|----------------|--------------|--------------------------------|
| Component      | Type         | Notes                          |
| employeeNumber | int(11)      | PK                             |
| lastName       | varchar(50)  |                                |
| firstName      | varchar(50)  |                                |
| extension      | varchar(10)  |                                |
| email          | varchar(100) |                                |
| officeCode     | varchar(10)  | FK → offices(officeCode)       |
| reportsTo      | Int(11)      | FK → employees(employeeNumber) |
| jobTitle       | varchar(50)  |                                |

Table 6. Offices Metadata

| Offices      |             |          |
|--------------|-------------|----------|
| Component    | Type        | Notes    |
| officeCode   | varchar(50) | PK       |
| city         | varchar(50) |          |
| phone        | varchar(50) |          |
| addressLine1 | varchar(50) |          |
| addressLine2 | varchar(50) | nullable |
| state        | varchar(50) | nullable |
| country      | varchar(50) |          |
| postalCode   | varchar(50) |          |
| territory    | varchar(50) |          |

Table 7. Orders Metadata

| Orders         |             |                                |
|----------------|-------------|--------------------------------|
| Component      | Type        | Notes                          |
| orderNumber    | int(11)     | PK                             |
| orderDate      | date        |                                |
| requiredDate   | date        |                                |
| shippedDate    | date        | nullable                       |
| status         | varchar(50) |                                |
| comments       | text        | nullable                       |
| customerNumber | Int(11)     | FK → customers(customerNumber) |

Table 8. Orderdetails Metadata

| Orderdetails    |               |                                |
|-----------------|---------------|--------------------------------|
| Component       | Type          | Notes                          |
| orderNumber     | int(11)       | PK, FK → orders(orderNumber)   |
| productCode     | varchar(50)   | PK, FK → products(productCode) |
| quantityOrdered | int(11)       |                                |
| priceEach       | decimal(10,2) |                                |
| orderLineNumber | smallint(6)   |                                |

Table 9. Payments Metadata

| Payments       |               |                                    |
|----------------|---------------|------------------------------------|
| Component      | Type          | Notes                              |
| customerNumber | int(11)       | PK, FK → customers(customerNumber) |
| checkNumber    | varchar(50)   | PK                                 |
| paymentDate    | date          |                                    |
| amount         | decimal(10,2) |                                    |

Table 10. Productlines Metadata

| Productlines    |               |          |
|-----------------|---------------|----------|
| Component       | Type          | Notes    |
| productLine     | varchar(50)   | PK       |
| textDescription | varchar(4000) | nullable |
| htmlDescription | mediumtext    | nullable |
| image           | mediumblob    | nullable |

Table 11. Products Metadata

| Products    |             |                                |
|-------------|-------------|--------------------------------|
| Component   | Type        | Notes                          |
| productCode | varchar(15) | PK                             |
| productName | varchar(15) |                                |
| productLine | varchar(15) | FK → productlines(productLine) |

|                    |               |  |
|--------------------|---------------|--|
| productScale       | varchar(15)   |  |
| productVendor      | varchar(15)   |  |
| productDescription | text          |  |
| quantityInStock    | smallint(6)   |  |
| buyPrice           | decimal(10,2) |  |
| MSRP               | decimal(10,2) |  |

### 5.2.3 How the System Uses Table Metadata in RAG, Validation, and Query Generation

Table metadata is what makes the whole Text-to-SQL pipeline work. It tells the system how to organize the data. When someone asks a question, ChromaDB stores and indexes the schema definitions. The retrieval module then picks the tables that are most relevant to the query based on how similar they are in meaning. This gives the LLM a focused schema context. This makes sure that the model only gets the metadata it needs to understand the structure of the database and not any other information that might be useful or misleading.

To make sure that the SQL that was generated is safe and correct, the same metadata is used during validation. The validator makes sure that all the tables and columns that are referenced in the schema are there. It also enforces strict SELECT-only rules to stop people from doing things they shouldn't. The LLM creates SQL using a controlled schema context, and the validator enforces these rules. This makes the final query more accurate, structurally sound, and in line with the real database.

## 5.3 Design Principles

The QueryMind user interface is easy to use, clear, and safe. The design uses a simple layout, easy-to-use navigation, and elements that respond to user's actions to make sure that both technical and non-technical users can access it. Only the most important information is shown, and error messages are clear but general to keep things safe.

### 5.3.1 Interface 1: Login Page

Users begin by entering their database credentials (host, port, username, password, database name) on a compact login form. The page highlights system features and security, with a welcoming introduction and tech stack badges.

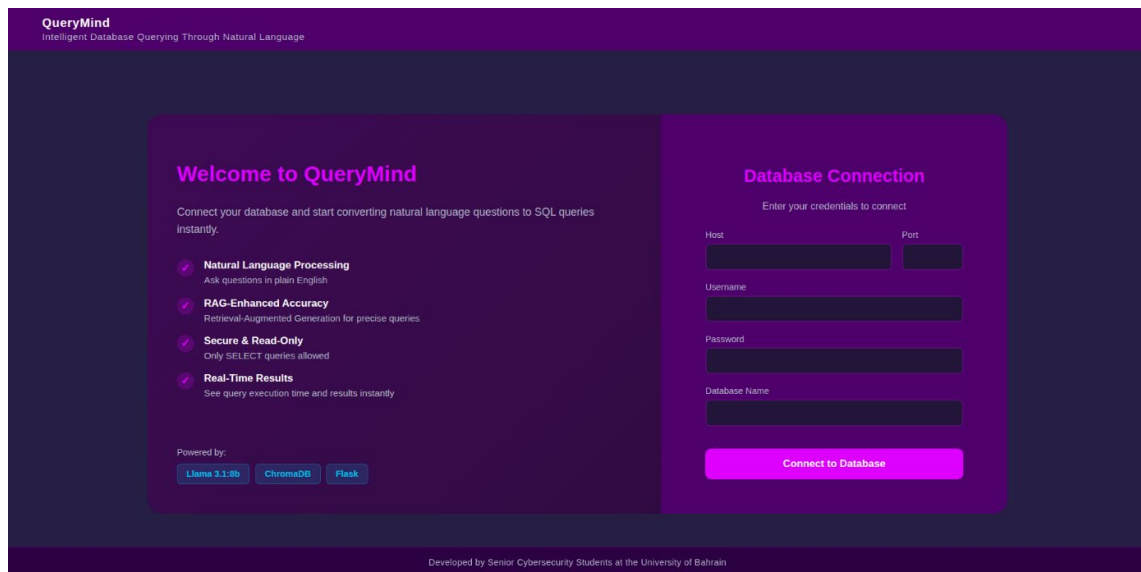


Figure 5. Login Page

### 5.3.2 Interface 2: Home Page

After you log in, you see *Figure 6. Home Page*. You use it to ask the database questions in simple terms. Type your question in the box in the middle. The system takes what you say and turns it into SQL, then gives you the results. The sidebar on the left shows all the tables you can use. The middle part gives you short examples to help you ask your questions. Only SELECT queries are allowed by the system. You can use this page to look at data, check records, or filter information without having to write SQL code yourself.

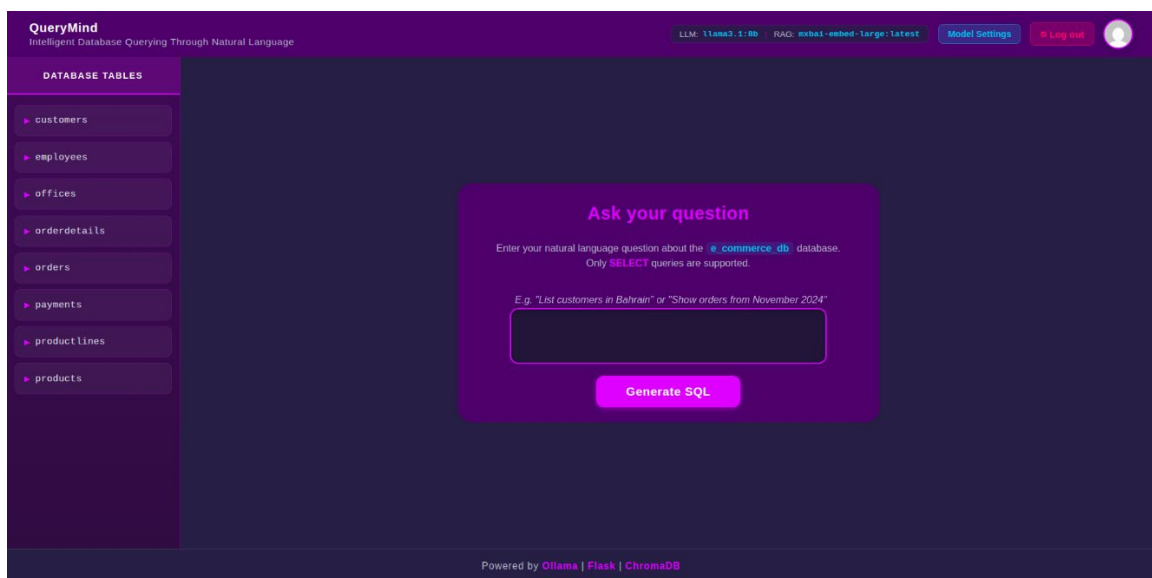
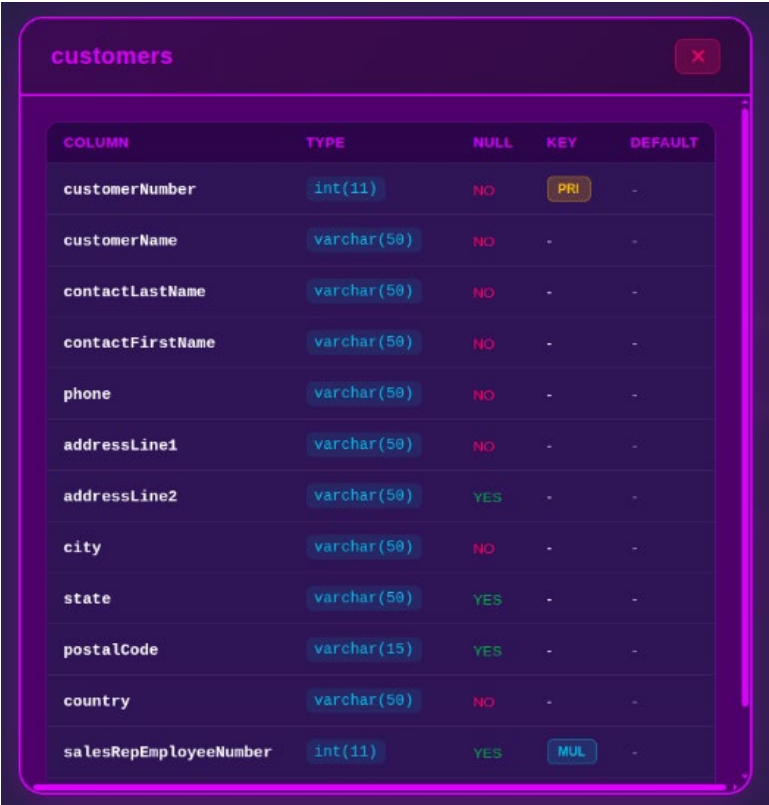


Figure 6. Home Page

The table box at the left panel enables you to view the tables columns and metadata, when clicked on any table, you see *Figure 7. Table Metadata*.



| COLUMN                 | TYPE        | NULL | KEY | DEFAULT |
|------------------------|-------------|------|-----|---------|
| customerNumber         | int(11)     | NO   | PRI | -       |
| customerName           | varchar(50) | NO   | -   | -       |
| contactLastName        | varchar(50) | NO   | -   | -       |
| contactFirstName       | varchar(50) | NO   | -   | -       |
| phone                  | varchar(50) | NO   | -   | -       |
| addressLine1           | varchar(50) | NO   | -   | -       |
| addressLine2           | varchar(50) | YES  | -   | -       |
| city                   | varchar(50) | NO   | -   | -       |
| state                  | varchar(50) | YES  | -   | -       |
| postalCode             | varchar(15) | YES  | -   | -       |
| country                | varchar(50) | NO   | -   | -       |
| salesRepEmployeeNumber | int(11)     | YES  | MUL | -       |

Figure 7. Table Metadata

5.3.3 Interface 3: Model Setting and Profile

On the *Figure 8. Navigation Bar*, we can see the current models used and a logout button. We can also see the profile page and see some information about username, host, port, and database that we’re currently connected with.

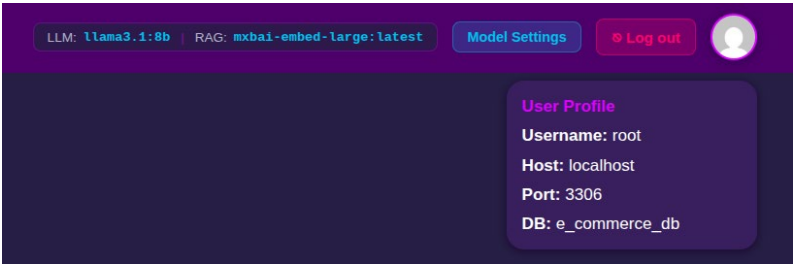


Figure 8. Navigation Bar

5.3.4 Interface 4: SQL Preview Panel

We asked about retrieving customer names with their total number of orders, so we are using JOIN here between 2 tables (customer and orders). Once the question is submitted, the system generates and displays the corresponding SQL query in a preview panel. This allows users to see the exact SQL that will be executed, along with timing metrics for each processing stage.



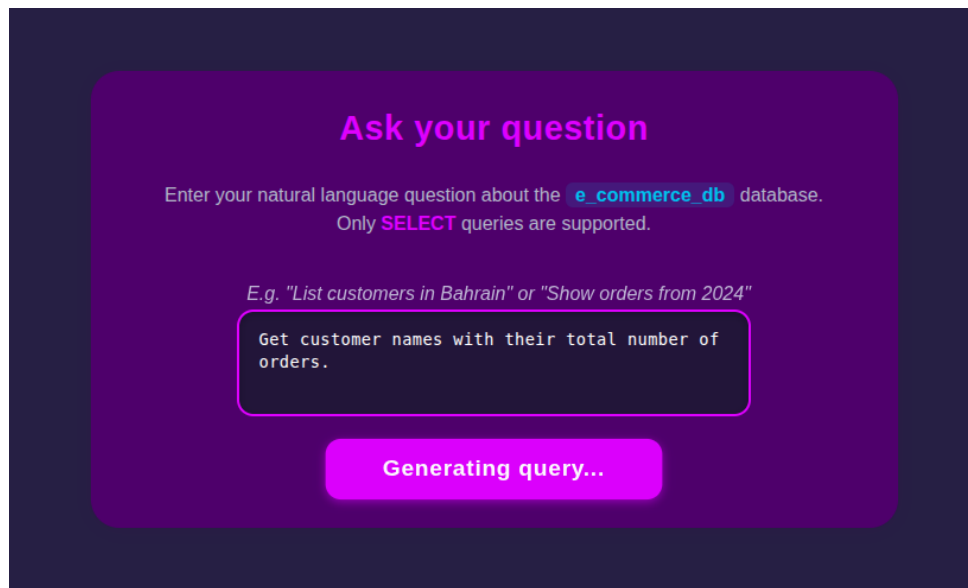


Figure 9. SQL Preview

### 5.3.5 Interface 5: Results Page

Figure 10. Result Page shows a complete breakdown of how the system dealt with the user's question. It shows the SQL query that was made, the time it took to get RAG, the time it took to make LLM, and the time it took to run the database. RAG retrieval shows how long it takes to get the schema context and relevant examples before asking the model to do something. LLM generation is the process of turning a natural language request into SQL with llama3.1:8b. Query execution tells you how long it took for the database to give back the results after the SQL was sent.

The total time reached about thirty-seven seconds. This is slower than ideal, but it is reasonable for a prototype built on a small eight-billion-parameter model without hardware optimization. The important part at this stage is the correctness of the generated SQL and the accuracy of the returned records. The results table shows that the system produced a valid joint and grouped the data correctly, which confirms that the model understood the question and generated an accurate query.

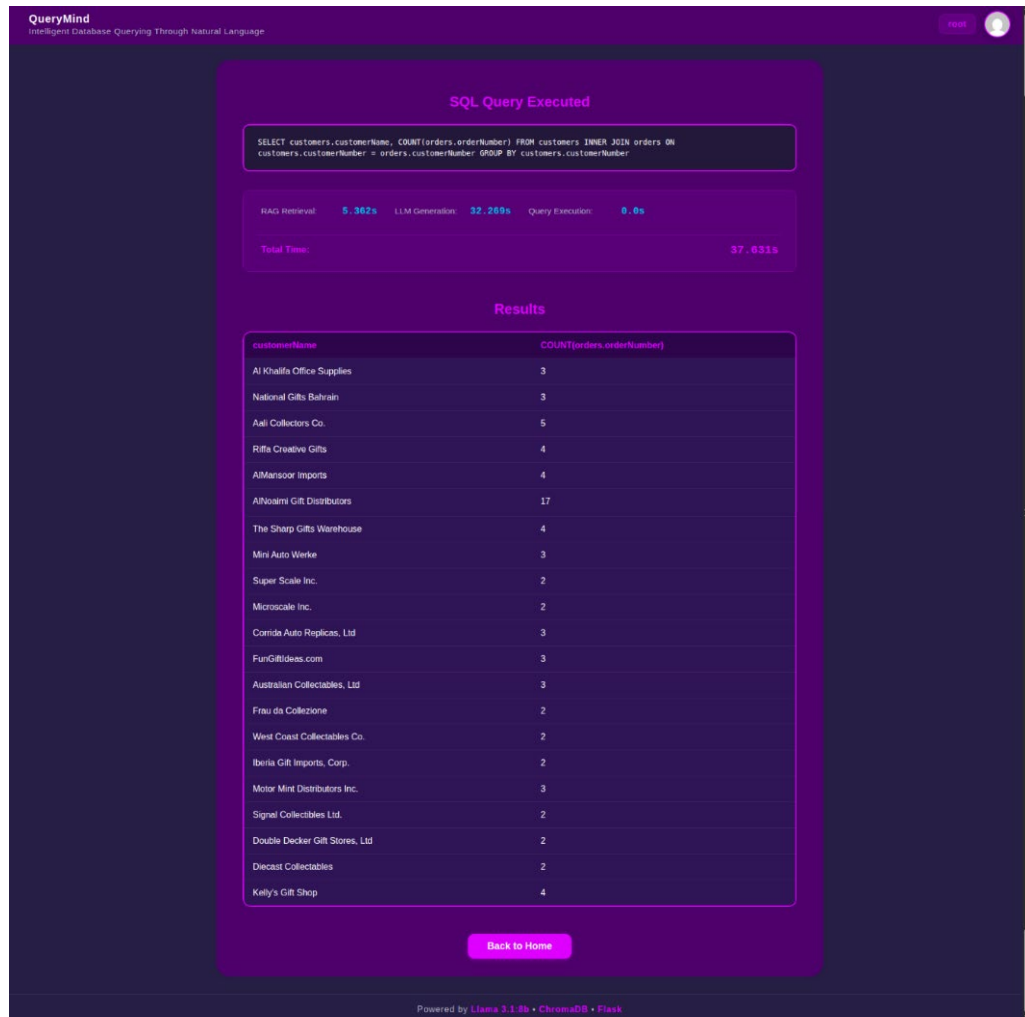


Figure 10. Result Page

### 5.3.6 Navigation and User Flow



Figure 11. User Flow

## 5.4 Core Procedures and Algorithms

This section goes into detail about the main steps and algorithms that make QueryMind's natural language to SQL conversion pipeline work. The pseudocode and implementation code for the main parts of the system are shown in each subsection. These parts include schema extraction, RAG-based context retrieval, prompt construction, LLM-based SQL generation, validation, and execution. To understand how user queries are turned into validated,

executable SQL statements while keeping security and data integrity intact, you need to know how these algorithms work.

### 5.4.1 Schema Extraction and Metadata Loading

The schema extraction module connects to the MariaDB database and retrieves the complete structure of all accessible tables. This metadata is essential for providing context to the LLM during query generation.

Pseudocode (app.py):

```
FUNCTION load_schema_from_session():
    connection = connect_to_database(session_credentials)
    IF connection is NULL:
        RETURN NULL

    tables = execute_query("SHOW TABLES")
    schema_text = ""

    FOR EACH table IN tables:
        create_statement = execute_query("SHOW CREATE TABLE table")
        schema_text += create_statement

    close_connection()
    RETURN schema_text, tables
```

This function iterates through all tables in the connected database, retrieves their CREATE TABLE statements, and concatenates them into a comprehensive schema text that describes the database structure.

### 5.4.2 RAG Retrieval Process

The Retrieval-Augmented Generation (RAG) process uses ChromaDB to store and retrieve relevant schema fragments based on semantic similarity to the user's question. This ensures the LLM receives only the most pertinent table definitions.

Pseudocode (chroma\_rag.py):

```
FUNCTION retrieve_schema_context(question, top_k):
    client = connect_to_chromadb(persist_path)
    collection = get_collection("db_schema")

    question_embedding = embed_text(question, model)
    results = query_collection(question_embedding, top_k)

    context_list = []
```

```

FOR EACH document, metadata IN results:
    context_list.append("Table: " + metadata.table_name + "\n" + document)

RETURN join(context_list)

```

The retrieval process converts the user's natural language question into a vector embedding, then queries the ChromaDB collection to find the most semantically similar table schemas, returning the top-k matches.

### 5.4.3 Prompt Construction Logic

The prompt construction module assembles a carefully structured prompt that includes the retrieved schema context, user question, and explicit instructions for the LLM to generate valid SQL.

Pseudocode (llm\_engine.py):

```

FUNCTION construct_prompt(question, rag_context):
    prompt = "You are an expert SQL assistant for MariaDB."
    prompt += "Generate a valid SQL SELECT query based on the user's question."
    prompt += "CRITICAL RULES: [validation rules]"
    prompt += "Database Schema: " + rag_context
    prompt += "User Question: " + question
    prompt += "SQL Query:"

    RETURN prompt

```

The prompt template enforces strict output formatting and includes security rules that prevent LLM from generating queries for unauthorized tables or non-SELECT operations.

### 5.4.4 SQL Generation Using LLM

The SQL generation module sends the constructed prompt to the Ollama LLM service and processes the response to extract a clean SQL query.

Pseudocode (llm\_engine.py):

```

FUNCTION generate_sql(question, rag_context, model_name):
    prompt = construct_prompt(question, rag_context)

    response = ollama.chat(model_name, prompt)
    sql_query = response.message.content

    sql_query = clean_response(sql_query) // Remove markdown, code blocks

    RETURN sql_query

```

The LLM engine communicates with the locally hosted Ollama service, receives the generated SQL, and performs post-processing to strip any formatting artifacts that the model may have included.

#### 5.4.5 SQL Validation Pipeline

The validation pipeline ensures that only SELECT queries are permitted, providing a security layer that prevents data modification or deletion operations.

Pseudocode (query\_executor.py):

```
FUNCTION extract_and_validate_sql(text):
    sql_lines = []
    in_sql_block = FALSE

    FOR EACH line IN text:
        IF line starts with "SELECT":
            in_sql_block = TRUE
        IF in_sql_block:
            sql_lines.append(line)

    extracted_sql = join(sql_lines)

    IF extracted_sql does not start with "SELECT":
        RETURN "Error: Only SELECT queries are allowed."

    RETURN extracted_sql
```

This validation function parses the LLM output, extracts SQL statements beginning with SELECT, and rejects any queries that do not conform to the read-only policy.

#### 5.4.6 Read-Only Execution Routine

The execution routine runs validated SELECT queries against the database and returns structured results including column names and row data.

Pseudocode (query\_executor.py):

```
FUNCTION run_query(sql_query, connection):
    cursor = connection.cursor()

    TRY:
        cursor.execute(sql_query)
        rows = cursor.fetchall()

    IF rows is empty:
        RETURN "No records."
```

```

column_names = get_column_names(cursor.description)
RETURN {columns: column_names, rows: rows}

```

```

CATCH error:
    RETURN "Error: " + error.message

```

```

FINALLY:
    cursor.close()

```

The execution routine returns results as a dictionary containing column headers and row data, enabling the frontend to render results in a tabular format.

### 5.4.7 Error Handling

The system implements comprehensive error handling at multiple levels to ensure graceful degradation and informative user feedback. Below are the pseudocode representations and corresponding implementations for each error handling scenario.

#### 1. Database Connection Error Handling

Pseudocode (app.py):

```

FUNCTION handle_login_error(exception):
    IF exception is MariaDB_Error:
        log_to_server("Login failed: " + exception.details)
        display_to_user("Invalid database credentials. Please check your information.")
    ELSE:
        log_to_server("Login error: " + exception.details)
        display_to_user("Connection failed. Please verify your credentials.")

    RETURN render_login_page(error_message)

```

#### 2. Schema Loading Error Handling

Pseudocode (schema\_loader.py):

```

FUNCTION handle_schema_error(exception):
    log_to_server("Schema load error: " + exception.message)
    RETURN error_string("Could not read schema from database: " + exception)

```

#### 3. RAG Retrieval Error Handling

Pseudocode (chroma\_rag.py):

```

FUNCTION handle_rag_error(exception):
    log_to_server("Error in retrieve_schema_context: " + exception.message)
    RETURN "Error retrieving schema context."

```

#### 4. LLM Generation Error Handling

Pseudocode (llm\_engine.py):

```

FUNCTION handle_llm_error(exception):
    RETURN "LLM Error: " + exception.message

```

#### 5. Query Execution Error Handling

Pseudocode (app.py):

```

FUNCTION handle_query_execution(sql_query, connection):
    TRY:
        start_time = current_time()
        results = run_query(sql_query, connection)
        execution_time = current_time() - start_time
    CATCH exception:
        error = "Error executing query: " + exception.message
        results = empty
        execution_time = 0
    FINALLY:
        connection.close()

    RETURN results, error, execution_time

```

## 6. ChromaDB Indexing Error Handling

Pseudocode (chroma\_rag.py):

```

FUNCTION handle_indexing_error(exception, persist_path):
    log_to_server("ChromaDB indexing error (non-fatal): " + exception.message)
    PASS

```

## 5.5 Mapping to Requirements

### 5.5.1 Functional Requirements Coverage

The design of the system meets all the functional needs listed in [Section 4.2](#). The query interface takes questions in plain English and shows the SQL that was created along with information on how it was run. This meets the need for easy input and clear feedback. The RAG pipeline provides schema-aware generation by getting only the table metadata that is needed and sending it to LLM. This makes sure that SQL matches the real structure of the database. The validation layer makes sure that only SELECT statements are allowed and blocks any unsafe ones. This keeps read-only access and meets basic security needs. The modular LLM engine and embedding configuration let you run the same tests on different model pairs repeatedly, using the same dataset and metrics. Using real database credentials also lets users see only the tables they are allowed to see, which makes dynamic visibility possible. Finally, the error-handling design gives clear and stable responses when there are problems with the connection, the schema, the retrieval, or the generation of invalid SQL. This meets the need for structured feedback.

### 5.5.2 Non-Functional Requirements Coverage

The system design meets security needs by only giving credentials to people who need them, blocking unsafe SQL patterns, and only allowing the LLM to access schema metadata instead of real records. Table metadata is the only information shared with the model, schema access

is based on user roles, and error messages don't give away internal details. This keeps confidentiality. Schema chunking, vector indexing, and smaller prompts all help with performance and scalability. They also keep response times reasonable even when using local models. A simple interface that shows examples, timing metrics, and results in a clear table layout makes the program easier to use. Modular parts for schema loading, retrieval, LLM generation, validation, and execution make the system easier to maintain. This means that new models or databases can be added without having to redesign the whole system.

## **5.6 Summary**

This chapter summarizes how QueryMind converts natural-language questions into safe SQL output. It outlined the role of table metadata in retrieval, validation, and correct SQL generation. It also showed the user flow from login to result display and described the core steps in schema extraction, RAG retrieval, prompt construction, SQL generation, and read-only execution. The chapter linked these design choices to the system requirements and highlighted how the design balances accuracy, speed, and security.



## Chapter 6

# System Implementation and Testing

This chapter presents the detailed implementation of QueryMind, describing how the system was developed from design to functional prototype. It covers the selection and integration of components, justification of implementation decisions, and provides a comprehensive walkthrough of the system's functionality. The chapter concludes with a thorough evaluation and testing phase, including security assessments and performance measurements.

### 6.1 System Implementation

This section describes the implementation details of QueryMind, including the development environment, technology stack selection, component integration, and system workflow demonstration.

#### 6.1.1 Development Environment and Hardware Specifications

The system was implemented and tested on a local development machine configured with the specifications shown in *Table 12. Hardware Configuration*. These hardware details provide a baseline for interpreting performance results, especially LLM generation time and overall pipeline latency. The environment ensures consistent and reproducible evaluation across all experiments.

*Table 12. Hardware Configuration*

| Component        | Specification              |
|------------------|----------------------------|
| Operating System | Linux (Ubuntu 24.04.3 LTS) |
| Processor        | AMD Ryzen 7 5800H          |
| RAM              | 16 GB DDR4                 |
| Storage          | 512 GB SSD                 |
| GPU              | NVIDIA GeForce RTX 3050    |

#### 6.1.2 Technology Stack Selection and Justification

The technologies used in QueryMind were selected to balance performance, security, and ease of integration. *Table 13. Technology Stack* summarizes the core components of the stack and

the reasoning behind each choice. Together, these tools enable a fully local, schema-aware Text-to-SQL system suitable for both testing and real deployment.

Table 13. Technology Stack

| Component          | Technology        | Justification                                                                                  |
|--------------------|-------------------|------------------------------------------------------------------------------------------------|
| Backend Framework  | Flask             | Lightweight, Python-native, easy integration with ML libraries, suitable for rapid prototyping |
| Database           | MariaDB/MySQL     | Open-source, widely adopted, SQL-standard compliant, supports both local and remote deployment |
| Database Connector | mariadb (Python)  | Native C connector, better performance than pure Python alternatives                           |
| LLM Runtime        | Ollama            | Local deployment, no API costs, privacy-preserving, supports multiple models                   |
| Primary LLM        | llama 3.1:8b      | Best accuracy in experiments (100%), open source                                               |
| Vector Database    | ChromaDB          | Lightweight, Python-native, persistent storage, cosine similarity support                      |
| Embedding Model    | mxbai-embed-large | High-quality embeddings, optimal for schema retrieval tasks                                    |
| Frontend           | HTML5/CSS3/JS     | No framework overhead, fast rendering, compatible with Flask templates                         |

### 6.1.3 Project Structure and Module Organization

```

Senior Project/
├── app.py                # Main Flask application and routing
├── chroma_rag.py         # RAG indexing and retrieval
├── llm_engine.py         # LLM prompt construction and inference
├── query_executor.py     # SQL validation and execution
├── schema_loader.py      # Database schema extraction
├── db_config.py          # Database configuration
├── main.py               # CLI interface (development)
├── templates/
│   ├── login.html       # Authentication interface
│   ├── home.html        # Query input interface
│   └── result.html       # Results display
├── static/
│   └── style.css         # Application styling
└── experiment/
    ├── exp_comp.py       # Experiment runner
    ├── aggregate_results.py # Results analysis
    └── gold_questions.json # Test cases

```

Figure 12. Project Structure

### 6.2 Evaluation and Testing

This section presents a comprehensive evaluation of QueryMind through multiple testing dimensions: security assessment, performance benchmarking, model comparison experiments,

deployment scenario analysis, and comparison with real solutions. The evaluation methodology ensures reliability and practical viability. The results are also compared with the real-world solution discussed in [Chapter 2.3](#) to measure how QueryMind performs against an existing system.

### 6.2.1 Testing Methodology

The evaluation framework uses a multi-layered testing approach to assess QueryMind’s functional accuracy, security resilience, and performance characteristics. A gold-standard dataset of five representative queries was used, each paired with its verified SQL statement and expected result set. The testing script automatically executes these queries, compares the system’s generated SQL with the gold standard, and determines whether the returned results match.

During each test run, the framework records context retrieval time, SQL generation time, and the exact query produced by the model. The same test suite is applied across all LLM and embedding model combinations to ensure consistent, repeatable benchmarking and fair comparison of performance and accuracy.

```
{
  "id": 1,
  "question": "List all customers that lives in France",
  "main_llm": "llama3.1:8b",
  "embedding_model": "mxbai-embed-large:latest",
  "gold_sql": "SELECT * FROM customers WHERE country = 'France'",
  "time_context_seconds": 0.043,
  "llm_generated_sql": "SELECT * FROM customers WHERE country = 'France'",
  "time_generate_sql_seconds": 27.201,
  "result_match": true
}

{
  "id": 2,
  "question": "Find product records that belong to the 'Trains' product line",
  "main_llm": "llama3.1:8b",
  "embedding_model": "mxbai-embed-large:latest",
  "gold_sql": "SELECT * FROM products WHERE productLine = 'Trains'",
  "time_context_seconds": 3.338,
  "llm_generated_sql": "SELECT * FROM products WHERE productLine = 'Trains'",
  "time_generate_sql_seconds": 25.019,
  "result_match": true
}

{
  "id": 3,
  "question": "Get payment records made by customer number 124",
  "main_llm": "llama3.1:8b",
  "embedding_model": "mxbai-embed-large:latest",
  "gold_sql": "SELECT * FROM payments WHERE customerNumber = 124",
  "time_context_seconds": 3.353,
  "llm_generated_sql": "SELECT * FROM payments WHERE customerNumber = 124",
  "time_generate_sql_seconds": 27.115,
  "result_match": true
}

{
  "id": 4
```

Figure 13. Gold Dataset Example

```

"question": "List all customers that lives in France",
"gold sql": "SELECT * FROM customers WHERE country = 'France'",
"expected rows": [
  [183, "Atelier graphique", "Schmitt", "Carine", "40.32.2555", "34", rue Royale, null, "Nantes", null, "44000", "France", 1370, "21000.00"],
  [119, "La Rochelle gifts", "Labrunie", "Janine", "47.09.8555", "67, rue des Cinqvingt etages", null, "Nantes", null, "44000", "France", 1370, "112800.00"],
  [146, "Savely & Henriot", "Co", "Savely", "Mary", "210.32.5555", "72, rue du Commerce", null, "Lyon", null, "69004", "France", 1337, "223900.00"],
  [171, "Daedalus Designs Imports", "Rance", "Martine", "20.10.1555", "104, chaussée de Tournai", null, "Lille", null, "59008", "France", 1370, "82900.00"],
  [102, "La Corne D'abondance", "Co", "Bertrand", "Marie", "11.42.34.2555", "265, boulevard Charonne", null, "Paris", null, "75012", "France", 1337, "8430.00"],
  [289, "Mini Caray", "Citeaux", "Frederique", "38.60.1555", "24, place Kleber", null, "Strasbourg", null, "67080", "France", 1370, "33800.00"],
  [124, "Alpha Copax", "Roulet", "Annette", "40.17.8555", "21 rue de Lorraine", null, "Toulouse", null, "31000", "France", 1370, "01100.00"],
  [256, "Lyon Souverains", "Do Silva", "Daniel", "133.14.62.7555", "27 rue du Colonel Pierre Avia", null, "Paris", null, "75508", "France", 1337, "68100.00"],
  [126, "Auto Associes & Cie.", "Tonini", "Daniel", "30.59.8555", "67, avenue de l'Europe", null, "Versailles", null, "78000", "France", 1370, "77900.00"],
  [530, "Marseille Mini Autos", "Lehrian", "Laurence", "91.24.4555", "32, rue des Bouchers", null, "Marseille", null, "13008", "France", 1337, "05900.00"],
  [142, "Alpha collectables", "Menciat", "Paul", "26.47.3555", "59 rue de l'Abbaye", null, "Reims", null, "51100", "France", 1337, "81100.00"],
  [406, "Auto Canal Petit", "Perrhan", "Dominique", "11.41.35.6555", "25, rue Lauriston", null, "Paris", null, "75016", "France", 1337, "95900.00"]
]

"question": "Find product records that belong to the 'Trains' product line",
"gold sql": "SELECT * FROM products WHERE productLine = 'Trains'",
"expected rows": [
  [510, 3259, "Collectable Wooden Train", "Trains", "118", "Carousell DieCast Legends, Hand crafted wooden toy train set is in about 1:18 scale.", "5320, 2027, 1950's Chicago Surface Lines Streetcar", "Trains", "132", "Gearbox Collectibles, This streetcar is a joy to see. It has 80 spare", "530, 1514, "Lacey City of Detroit Streetcar", "Trains", "130", "Classic Metal Creations", This streetcar is a joy to see. It has 99 separate
]

"question": "Get payment records made by customer number 124",
"gold sql": "SELECT * FROM payments WHERE customerNumber = 124",
"expected rows": [
  [124, "2018-03-31", "2005-03-05", "10124.50"],
  [124, "2005-03-04", "2004-08-28", "85410.87"],
]

```

Figure 14. Results Example

Table 14. Test Dataset

| Query ID | Type                  | Complexity | Description                       |
|----------|-----------------------|------------|-----------------------------------|
| Q1       | Simple SELECT         | Low        | Filter customers by country       |
| Q2       | Filtered SELECT       | Low        | Filter products by category       |
| Q3       | Filtered SELECT       | Low        | Filter payments by customer ID    |
| Q4       | JOIN with Filter      | Medium     | Multi-table join with date filter |
| Q5       | JOIN with Aggregation | High       | Group by with count aggregation   |

The evaluation relies on five key metrics. The result match rate measures how many generated queries return the correct results compared to the gold standard. Context retrieval time records how long RAG takes to identify the relevant schema segments. SQL generation time captures the duration required by the LLM to produce the query. Query execution time measures how long the database takes to run the validated SQL. Finally, total pipeline time represents the combined duration of all processing stages from input to output.

### 6.2.2 Security Testing

Security testing was conducted to validate our system defense mechanisms against common attack vectors. All tests were performed in a controlled environment; results were included below.

Table 15. Security Testing Results

| Test ID | Testing Criteria      | Target                      | Test Input                                       | Result | Explanation                                                                   |
|---------|-----------------------|-----------------------------|--------------------------------------------------|--------|-------------------------------------------------------------------------------|
| SEC-01  | SQL Injection (Login) | Login form - Username field | ' or 1=1 --                                      | Pass   | Credentials validated against database; injection string fails authentication |
| SEC-02  | SQL Injection (Query) | Query input textarea        | `; DROP TABLE customers; --`                     | Pass   | SELECT-only validation rejects non-SELECT statements                          |
| SEC-03  | SQL Injection (Query) | Query input textarea        | `UNION SELECT * FROM users`                      | Pass   | Query blocked; only SELECT queries from user question allowed                 |
| SEC-04  | XSS Attack            | Query input textarea        | `<script>alert('XSS')</script>`                  | Pass   | Jinja2 auto-escaping prevents script execution                                |
| SEC-05  | Session Hijacking     | Client Cookie               | Attempt to take over a user session after logout | Pass   | Cookie is deleted when the session ends                                       |

Figure 15. SEC-01 shows the system rejecting a SQL injection attempt during login. The injected username fails authentication, demonstrating that credentials are validated securely and unsafe input is blocked.

The screenshot shows a 'Database Connection' form with the following fields and values:

- Host: localhost
- Port: 3306
- Username: ' or 1=1 --
- Password: (masked with dots)
- Database Name: e\_commerce\_db

Below the fields is a red error message: 'Invalid database credentials. Please check your information and try again.' At the bottom is a 'Connect to Database' button.

Figure 15. SEC-01

Figure 16. SEC-02 and Figure 17. SEC-03 show the system blocking SQL injection attempts entered through the query field. In SEC-02, the malicious DROP TABLE command is rejected by the SELECT-only validator. In SEC-03, the UNION SELECT injection attempt is also denied, since the system restricts generated queries to the tables and columns derived from the user’s input. Together, these figures demonstrate that injection-based attacks cannot bypass the validation layer.

The screenshot shows the 'Ask your question' form with the following details:

- Header: Ask your question
- Text: Enter your natural language question about the e\_commerce\_db database. Only SELECT queries are supported.
- Example: E.g. "List customers in Bahrain" or "Show orders from 2024"
- Input field: ; DROP TABLE customers; --
- Button: Generate SQL
- Footer: Error: Only SELECT queries are allowed.

Figure 16. SEC-02

The screenshot shows the 'Ask your question' form with the following details:

- Header: Ask your question
- Text: Enter your natural language question about the e\_commerce\_db database. Only SELECT queries are supported.
- Example: E.g. "List customers in Bahrain" or "Show orders from 2024"
- Input field: UNION SELECT \* FROM users
- Button: Generate SQL
- Footer: Error: Only SELECT queries are allowed.

Figure 17. SEC-03

Figure 18. SEC-04 shows the system preventing execution of a cross-site scripting payload. The injected <script> tag safely escaped by the template engine, demonstrating protection against XSS attacks.

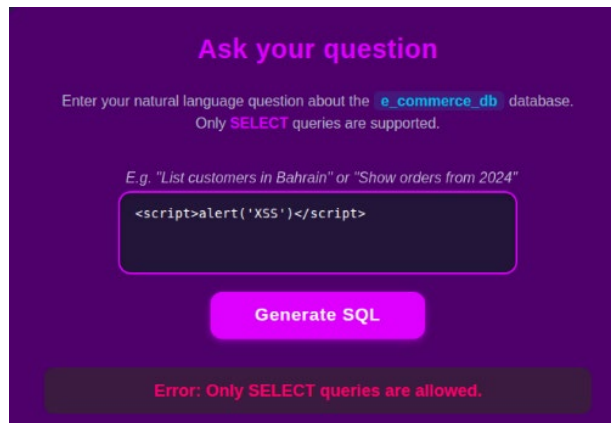


Figure 18. SEC-04

Figure 19. SEC-05 the system invalidating the session cookie upon logout. Attempts to reuse the expired cookie will fail, confirming protection against session hijacking.

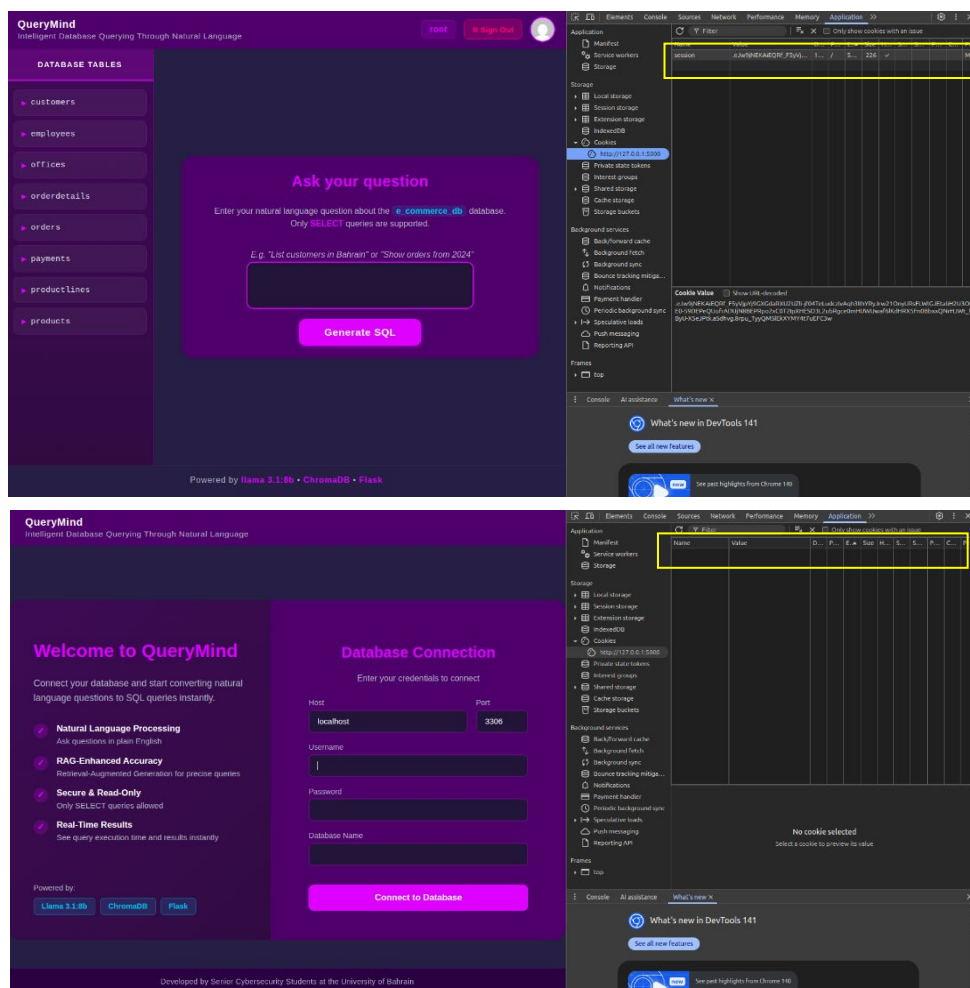


Figure 19. SEC-05

## 6.2.3 Performance Measurements

Performance testing measured the execution time of core system functions to assess scalability and identify bottlenecks.

Table 16. Core Function Performance Metrics

| Function                  | Description                  | Avg Time (s)  | Min Time (s)  | Max Time (s)  | % of Total  |
|---------------------------|------------------------------|---------------|---------------|---------------|-------------|
| retrieve_schema_context() | RAG retrieval from ChromaDB  | 4.319         | 0.043         | 5.438         | 12.9%       |
| ask_llm()                 | LLM SQL generation           | 28.974        | 25.019        | 34.106        | 86.6%       |
| extract_sql()             | SQL validation               | <0.001        | <0.001        | <0.001        | <0.1%       |
| run_query()               | Database execution           | 0.003         | 0.000         | 0.013         | <0.1%       |
| <b>Total Pipeline</b>     | <b>End-to-end processing</b> | <b>33.459</b> | <b>30.505</b> | <b>39.587</b> | <b>100%</b> |

Key Performance: LLM generation takes the largest share of processing time at 86.6 percent, so model inference is the main bottleneck. RAG retrieval adds 12.9 percent due to the *mxbai-embed-large* model. Database execution and SQL validation add almost no delay, with execution staying below 0.01 seconds.

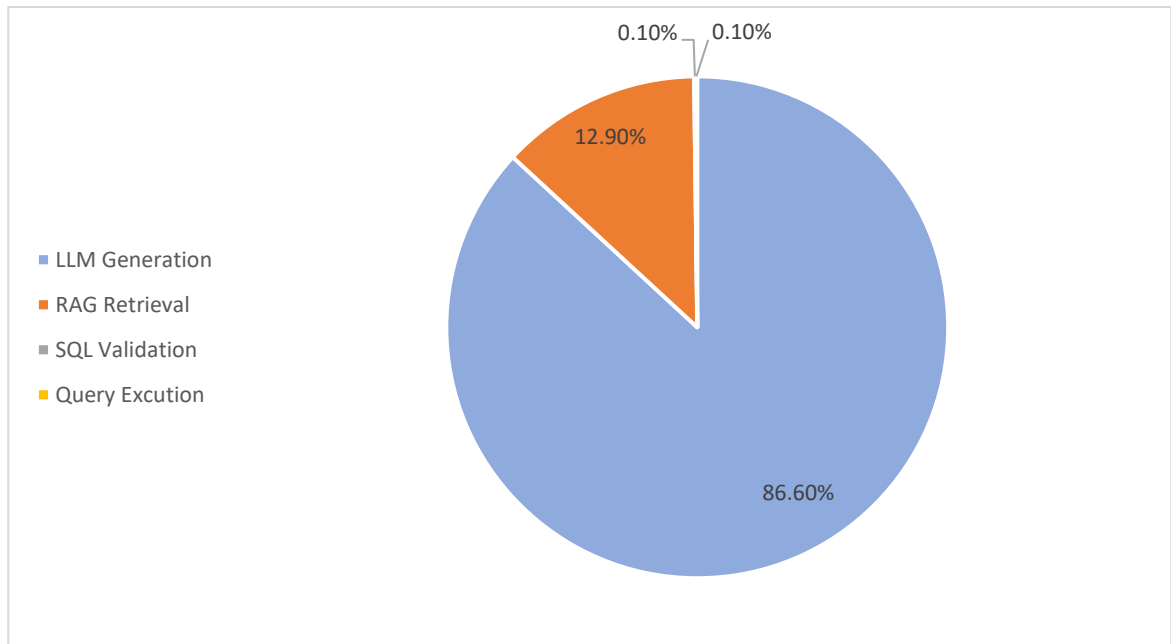


Figure 20. Time Distribution Pie Chart

### 6.2.4 Model Comparison Experiments

Comprehensive experiments were conducted to evaluate different combinations of LLM and embedding models. Five configurations were tested against the gold standard dataset. *Table 17. Model Combination Result* summarizes the performance of all five model configurations, comparing match rate, context retrieval time, SQL generation time, and total pipeline duration for each setup.

Table 17. Model Combination Result

| Configuration | LLM Model   | Embedding Model   | Match Rate | Avg Context (s) | Avg Generation (s) | Total Avg (s) |
|---------------|-------------|-------------------|------------|-----------------|--------------------|---------------|
| Config 1      | llama3.1:8b | mxbai-embed-large | 100%       | 4.319           | 28.974             | 33.293        |
| Config 2      | qwen3:1.7b  | mxbai-embed-large | 60%        | 0.150           | 6.686              | 6.836         |
| Config 3      | gemma3:1b   | mxbai-embed-large | 60%        | 0.141           | 5.193              | 5.334         |
| Config 4      | qwen3:1.7b  | all-minilm        | 60%        | 0.032           | 4.515              | 4.547         |
| Config 5      | gemma3:1b   | all-minilm        | 40%        | 0.032           | 4.203              | 4.235         |

*Figure 21. Model Accuracy* illustrates the result match rate achieved by each configuration, showing how accuracy varies depending on the selected LLM and embedding model.

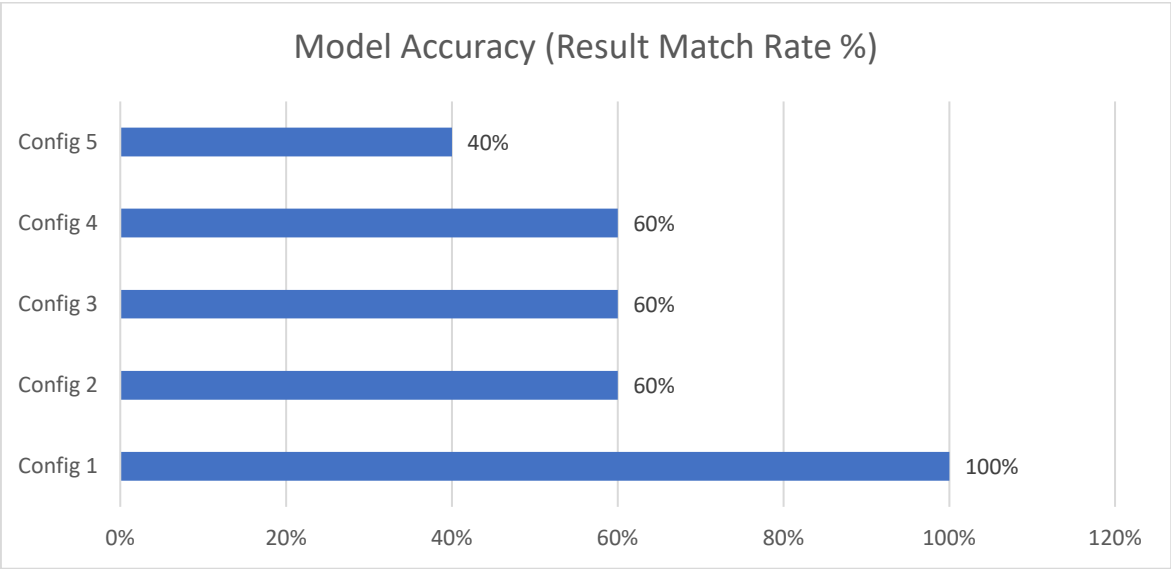


Figure 21. Model Accuracy

*Figure 22. Average SQL Generation Time* presents the average SQL generation time for each configuration, highlighting the trade-off between model size and response speed.



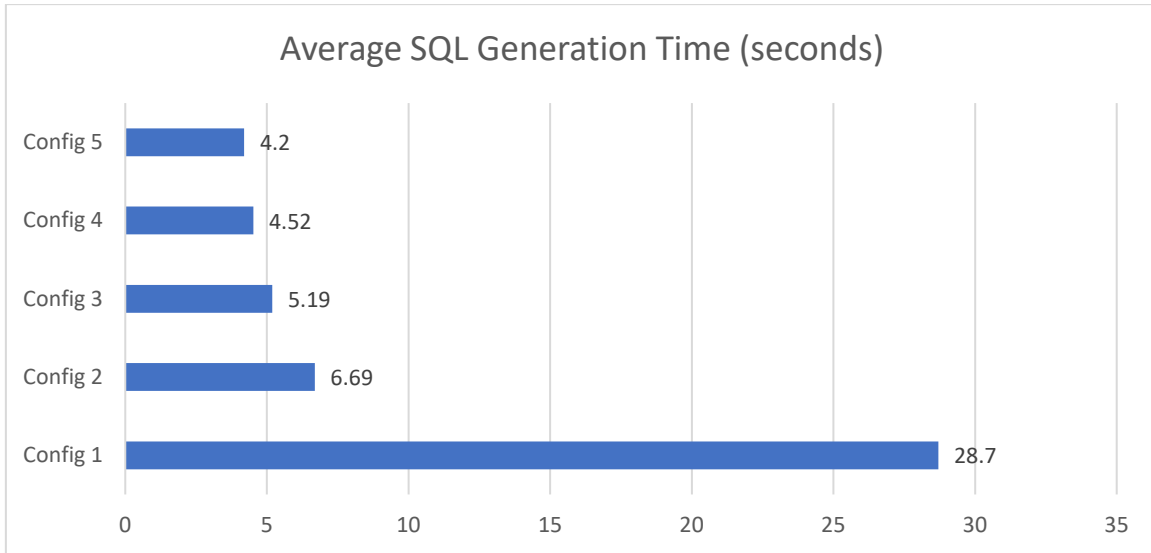


Figure 22. Average SQL Generation Time

This table shows how each model configuration performs across different query types, indicating which models succeed or fail on simple, filtered, and multi-table JOIN queries.

Table 18. Query Complexity Analysis

| Query Type                 | llama3.1:8b | qwen3.1:7b | gemma3.1:1b |
|----------------------------|-------------|------------|-------------|
| Simple SELECT (Q1-Q3)      | 3/3         | 3/3        | 3/3         |
| JOIN with Filter (Q4)      | 1/1         | 0/1        | 0/1         |
| JOIN with Aggregation (Q5) | 1/1         | 0/1        | 0/1         |

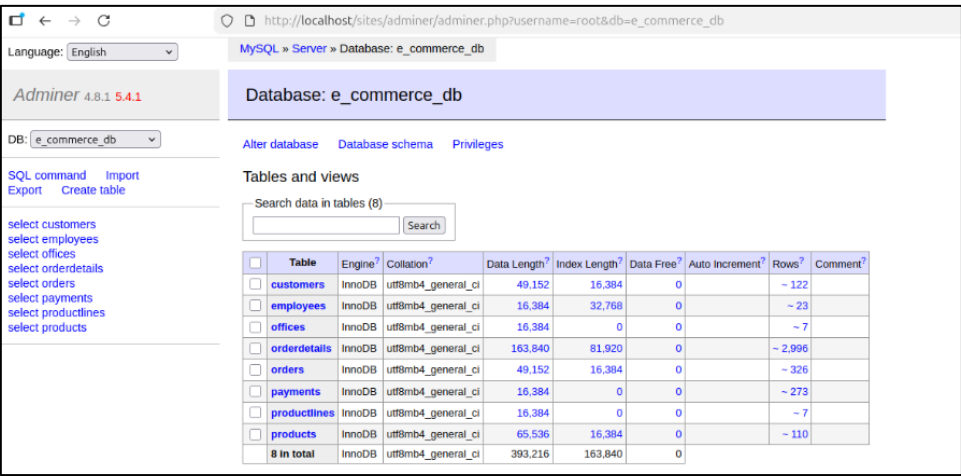
**Key Findings:** The results show that accuracy increases with larger models. The *8b* model produced correct SQL for all queries but required more processing time. Smaller models generated responses much faster, yet often failed on JOIN and aggregation queries, sometimes producing incorrect column names or substituting JOINs with subqueries. Across all configurations, embedding models showed consistently fast retrieval times, meaning they did not contribute to latency. The main factor affecting total runtime was the LLM itself, with larger models trading speed for accuracy and smaller models trading accuracy for speed. Overall, *llama3.1-8b* with *mxbai-embed-large* delivered the most reliable performance.

### 6.2.5 Local vs Remote Database Performance

To assess deployment flexibility, QueryMind was tested against both local and remote database configurations using the optimal model combination (*llama3.1:8b* + *mxbai-embed-large*).

### 6.2.5.1 Test Configuration

Local: MariaDB on localhost (0ms network latency)



| Table        | Engine | Collation          | Data Length | Index Length | Data Free | Auto Increment | Rows    | Comment |
|--------------|--------|--------------------|-------------|--------------|-----------|----------------|---------|---------|
| customers    | InnoDB | utf8mb4_general_ci | 49,152      | 16,384       | 0         |                | ~ 122   |         |
| employees    | InnoDB | utf8mb4_general_ci | 16,384      | 32,768       | 0         |                | ~ 23    |         |
| offices      | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 0         |                | ~ 7     |         |
| orderdetails | InnoDB | utf8mb4_general_ci | 163,840     | 81,920       | 0         |                | ~ 2,996 |         |
| orders       | InnoDB | utf8mb4_general_ci | 49,152      | 16,384       | 0         |                | ~ 326   |         |
| payments     | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 0         |                | ~ 273   |         |
| productlines | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 0         |                | ~ 7     |         |
| products     | InnoDB | utf8mb4_general_ci | 65,536      | 16,384       | 0         |                | ~ 110   |         |
| 8 in total   | InnoDB | utf8mb4_general_ci | 393,216     | 163,840      | 0         |                |         |         |

Figure 23. Local DB via Adminer

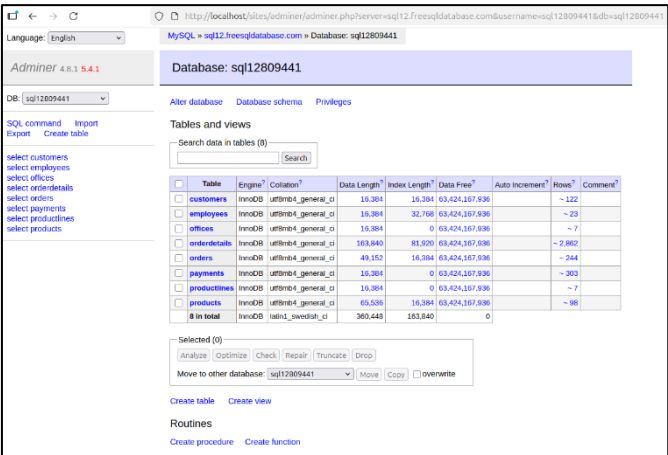
Remote: MySQL via www.freesqldatabase.com (50-200ms network latency)

| Account Details |                     |                 |
|-----------------|---------------------|-----------------|
| Account Number  | Available Databases | Available Space |
| 1532024         | 0 of 1              | 10% of 5.00MB   |

| Database Details          |               |                   |                   |        |        |                                  |
|---------------------------|---------------|-------------------|-------------------|--------|--------|----------------------------------|
| Database Host             | Database Name | Database Username | Database Password | Size   | Status | Delete                           |
| sql12.freesqldatabase.com | sql12809441   | sql12809441       | Check your emails | 0.50MB | Live   | <input type="checkbox"/>         |
|                           |               |                   |                   |        |        | <button>Delete database</button> |

Figure 24. Remote DB Credentials



| Table        | Engine | Collation          | Data Length | Index Length | Data Free      | Auto Increment | Rows    | Comment |
|--------------|--------|--------------------|-------------|--------------|----------------|----------------|---------|---------|
| customers    | InnoDB | utf8mb4_general_ci | 16,384      | 16,384       | 63,424,167,936 |                | ~ 122   |         |
| employees    | InnoDB | utf8mb4_general_ci | 16,384      | 32,768       | 63,424,167,936 |                | ~ 23    |         |
| offices      | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 63,424,167,936 |                | ~ 7     |         |
| orderdetails | InnoDB | utf8mb4_general_ci | 163,840     | 81,920       | 63,424,167,936 |                | ~ 2,982 |         |
| orders       | InnoDB | utf8mb4_general_ci | 49,152      | 16,384       | 63,424,167,936 |                | ~ 244   |         |
| payments     | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 63,424,167,936 |                | ~ 303   |         |
| productlines | InnoDB | utf8mb4_general_ci | 16,384      | 0            | 63,424,167,936 |                | ~ 7     |         |
| products     | InnoDB | utf8mb4_general_ci | 65,536      | 16,384       | 63,424,167,936 |                | ~ 98    |         |
| 8 in total   | InnoDB | utf8mb4_general_ci | 360,448     | 163,840      | 0              |                |         |         |

Figure 25. Remote DB via Adminer

### 6.2.5.2 Performance Comparison

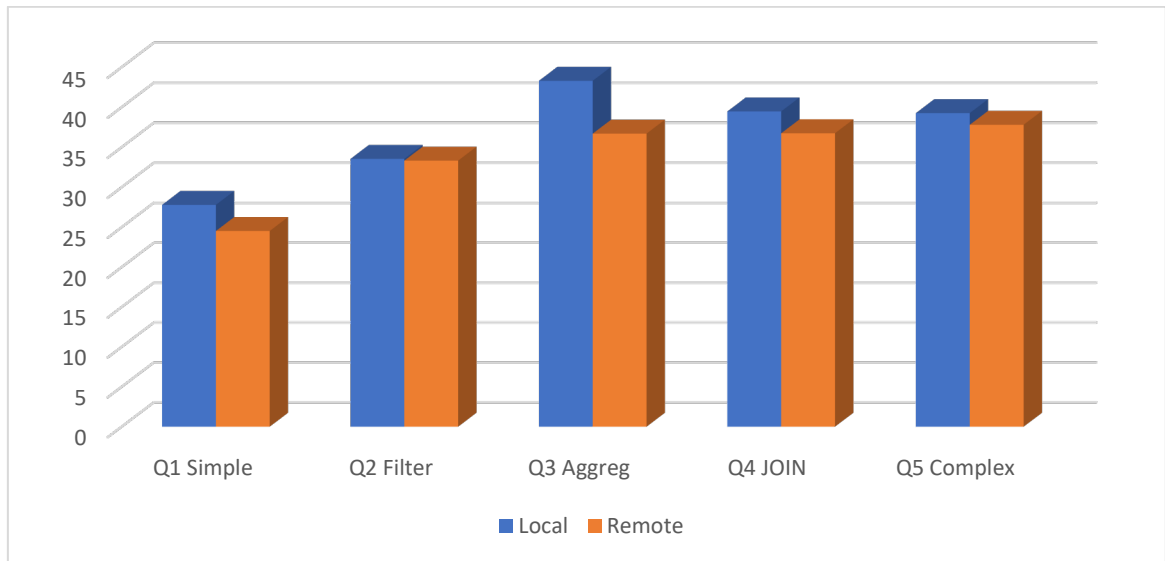
This table compares total pipeline time and raw SQL execution time between local and remote database deployments across all five test queries. It shows how database location influences overall performance.

Table 19. Local vs Remote Performance Comparison

| Query               | Local Total (s) | Remote Total (s) | Local Exec (s) | Remote Exec (s) |
|---------------------|-----------------|------------------|----------------|-----------------|
| Q1: Simple SELECT   | 27.79           | 24.53            | 0.000          | 0.148           |
| Q2: Filtered SELECT | 33.54           | 33.32            | 0.000          | 0.147           |
| Q3: Aggregation     | 42.30           | 36.70            | 0.003          | 0.140           |

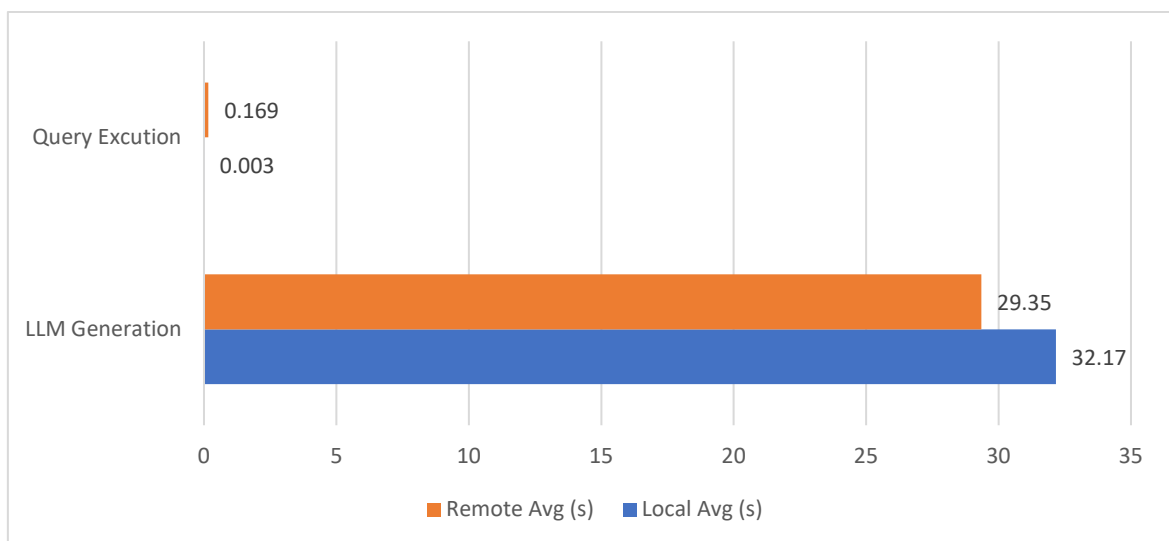
|                   |              |              |              |              |
|-------------------|--------------|--------------|--------------|--------------|
| Q4: JOIN + Filter | 39.47        | 36.74        | 0.000        | 0.147        |
| Q5: Multi-JOIN    | 39.26        | 37.80        | 0.013        | 0.262        |
| <b>Average</b>    | <b>36.47</b> | <b>33.82</b> | <b>0.003</b> | <b>0.169</b> |

*Figure 26. Local vs Remote Total Generation Time* visualizes the total generation time for each query under local and remote setups, illustrating how both environments perform across simple, filtered, and complex queries.



*Figure 26. Local vs Remote Total Generation Time*

*Figure 27. Time Component Breakdown* shows the average time spent on LLM generation and SQL execution for local and remote deployments, highlighting which parts of the pipeline contribute most to overall latency.



*Figure 27. Time Component Breakdown*

The results show that remote deployment was a little faster, but this was due to normal LLM generation variance and no differences in the database. Network latency didn't matter because database execution is only a small part of the pipeline. The extra delay in remote mode was not significant compared to the total runtime. Most of the time was spent generating the LLM, so where the database is located doesn't affect performance. Both deployments reached full accuracy in about the same amount of time, which shows that QueryMind works well in both setups.

### 6.2.6 Comparison with Real Solution

To benchmark QueryMind against a real system, we compared it with text2sql.ai [6], a commercial text-to-SQL platform. We used the same natural language questions for both systems. Because text2sql.ai operates on PostgreSQL and QueryMind uses MySQL, we assessed correctness based on the logical results rather than the SQL syntax itself. The goal was to measure accuracy and output quality under equal conditions. Speed was also considered and as expected, text2sql.ai responded faster because it runs on a fully optimized cloud backend with dedicated compute, while QueryMind processes all LLM inference locally on CPU. This created a natural speed gap, but it did not affect the fairness of the accuracy comparison.

*Table 20. QueryMind vs text2sql.ai Comparison* compares QueryMind and text2sql.ai using the same set of five benchmark queries. It shows whether each system produced correct SQL and highlights the differences in accuracy between both tools.

*Table 20. QueryMind vs text2sql.ai Comparison*

| Query | Question                                       | QueryMind    | text2sql.ai | Match |
|-------|------------------------------------------------|--------------|-------------|-------|
| 1     | List customers with phone starting with '3555' | Correct      | Correct     | Yes   |
| 2     | Show product names and their categories        | Correct      | Correct     | Yes   |
| 3     | List orders placed in November 2025            | Correct      | Correct     | Yes   |
| 4     | Show order items for customer 'John Doe'       | Schema Error | Correct     | No    |
| 5     | Total payments received for November 2025      | Correct      | Correct     | Yes   |

*Figure 28. Accuracy Comparison* visualizes the accuracy results from Table 22, comparing the percentage of correct outputs generated by QueryMind and text2sql.ai across the shared test set.

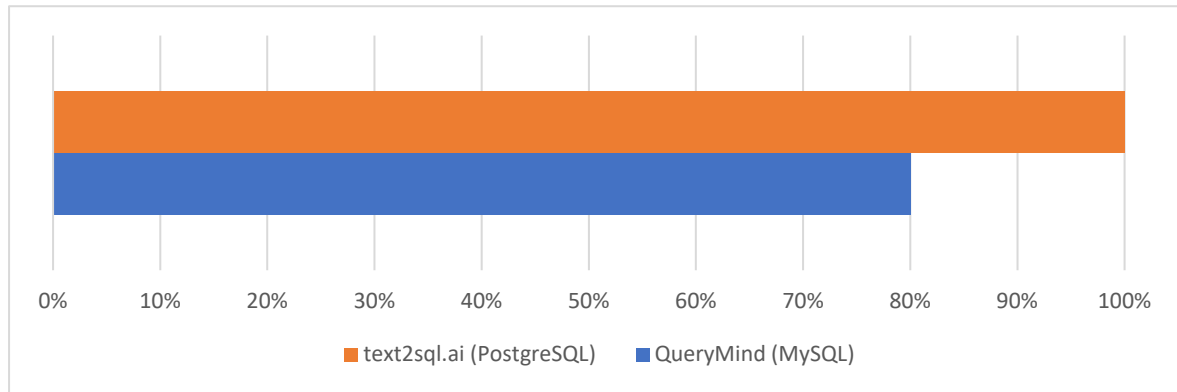


Figure 28. Accuracy Comparison

This table provides a detailed feature-by-feature comparison of QueryMind and text2sql.ai, covering accuracy, deployment model, privacy controls, database support, customization, and additional capabilities [6].

Table 21. Feature Comparison Matrix

| Feature              | QueryMind                                  | text2sql.ai                                                             |
|----------------------|--------------------------------------------|-------------------------------------------------------------------------|
| Accuracy Rate        | 80% on our test set                        | 100% on the same questions                                              |
| Cost                 | Free, open-source                          | Subscription plans with a free trial and request-based quotas           |
| Data Privacy         | All data, schema, and LLM calls stay local | Hosted service and API with documented security and privacy controls    |
| Deployment           | Self-hosted Flask web app, local LLM       | SaaS web app, desktop client, and public API for integration            |
| Database Support     | MySQL/MariaDB                              | Multiple SQL dialects (PostgreSQL, MySQL, SQL Server, Oracle) and more  |
| Schema Handling      | RAG over live schema extracted at login    | Connection wizard with automatic schema extraction and review           |
| Collaboration        | Single-user prototype                      | Teams, shared connections, role-based access, seat billing              |
| Extra AI Features    | Text-to-SQL only                           | Explain, fix, and optimize SQL; Excel/Sheets formulas; regex generation |
| Internet Required    | No, fully local (LLM and DB)               | Yes for SaaS/API; desktop app uses provider models in the background    |
| API and Integrations | None in current prototype                  | Production API for programmatic use and custom tools                    |

### 6.2.6.1 Analysis

1. Accuracy: text2sql.ai answered all questions correctly. QueryMind missed one due to a column-name mismatch, which points to a schema handling issue rather than a limitation in the LLM or RAG approach.
2. Feature Depth: text2sql.ai offers broader functionality such as SQL explanation, fixing, optimization, and multi-dialect support. QueryMind focuses on text-to-SQL with strong privacy and local control.
3. Performance: text2sql.ai delivers faster responses due to its optimized cloud and desktop infrastructure. QueryMind runs fully on local CPU, which increases latency but improves privacy.
4. Security and Control: QueryMind keeps all data and inference local, which suits sensitive environments. text2sql.ai relies on external infrastructure but offers enterprise-ready workflow features.
5. Positioning: text2sql.ai is a mature commercial product. QueryMind is a research prototype that provides competitive accuracy while remaining open-source, customizable, and privacy-focused.

### 6.2.7 Strength and weaknesses Analysis

This table summarizes the main strengths of QueryMind and links each strength to evidence gathered from earlier experiments and evaluations. It highlights where the system performs well, including accuracy, privacy, governance controls, and deployment flexibility.

Table 22. System Strength

| Strength            | Description                                                        | Evidence                                         |
|---------------------|--------------------------------------------------------------------|--------------------------------------------------|
| Secure by Design    | SELECT-only enforcement, schema validation, no access to live data | Table 15. Security Testing Results               |
| Local Privacy       | Fully offline LLM inference and schema handling                    | No external API calls or cloud storage           |
| High Accuracy       | Consistent 100% result match with top model (Llama3.1 + mxbai)     | Table 17. Model Combination Result               |
| Model Flexibility   | Supports multiple LLM and embedding combinations                   | Table 17. Model Combination Result               |
| Deployment Ready    | Runs equally on local and remote databases                         | Table 19. Local vs Remote Performance Comparison |
| Scalable Schema Use | Scales to many tables without context overflow                     | Per-table chunking strategy                      |
| Open-Source         | Full customization                                                 | Complete source code available                   |

Table 23. *System Weaknesses* presents the key weaknesses identified during testing and usage, along with potential mitigation strategies. This helps clarify current system limitations and outlines actionable improvements for future development.

Table 23. *System Weaknesses*

| Weakness                 | Description                           | Potential Mitigation                                         |
|--------------------------|---------------------------------------|--------------------------------------------------------------|
| Slow LLM Inference       | 25-30 seconds per query               | GPU acceleration                                             |
| No Write Support         | Only SELECT queries allowed by design | Add safe INSERT/UPDATE/DELETE pipeline in future work        |
| Single Database Session  | Cannot query across databases         | Reauthentication required for different DB                   |
| Limited DB Type          | Support only MySQL                    | Extend SQL dialect handling for PostgreSQL, SQL Server, etc. |
| Complex Query Limitation | Smaller Models struggle with JOINS    | Use larger models like <i>llama3.1:8b</i>                    |
| No Query History         | Queries not persisted                 | Add session-based history feature                            |

### 6.2.8 Discussion and Comparison with Literature

The results of this study demonstrate that QueryMind aligns with contemporary trends in research pertaining to secure and schema-governed Text-to-SQL systems. Recent analyses in the industry highlight the increasing demand for systems that safeguard data confidentiality while facilitating intelligent data access, particularly as organizations transition to enterprise-wide data-driven operations [1], [2]. [Chapter 2](#) review of the literature shows a shift towards metadata-restricted generation. QueryMind follows this trend by only showing schema information to LLM. This is like Exasol's [3] governed SQL approach. This design choice directly improves auditability and controlled execution, which are very important in places where privacy and governance rules are very strict.

Recent academic study underscores that Text-to-SQL evaluation should prioritize execution accuracy over syntactic similarity. This is in line with the execution-driven methodology that Zhai et al. proposed, which showed that correctness should be based on real query results rather than SQL string matching [4]. QueryMind uses this idea by comparing the returned table to a known gold standard. This makes it possible to get a more realistic and useful measure of how reliable the model is across different configurations.

The system also shows the architectural ideas talked about in big business solutions like LinkedIn's metadata-driven workflow [5]. QueryMind benefits from structured schema

retrieval in the same way that their approach does. This shows how the quality of metadata and the completeness of schemas affect the accuracy of LLMs. Our results show that smaller models have trouble with multi-table JOINS and aggregation tasks, just like LinkedIn's deployment. This is a well-known problem in production-level Text-to-SQL literature.

When compared to the commercial tool `text2sql.ai`, the real-world trade-offs between managed SaaS platforms and open-source systems that are installed on-site become even clearer [6]. `Text2sql.ai` has more SQL engine support and extra features like query optimization and APIs, but you must trust third-party infrastructure and pay for access. Instead, QueryMind gives you full control over your data, schema, and model execution on your own computer. This keeps your data private while allowing you to customize and experiment. This makes it useful in places where privacy is important, like finance, government, or businesses that must follow rules.

Overall, QueryMind fills in a lot of the holes that have been found in literature, especially when it comes to governance, deployment control, and SQL generation that can be audited. Even though it runs slower than proprietary systems, it can still get competitive accuracy when given high-quality schema metadata. This shows that secure, fully local Text-to-SQL systems can work well in the real world.



## Chapter 7

# Conclusion and Future Work

QueryMind gave us a full text-to-SQL system that makes accurate and safe SQL queries from questions in natural language. The system protects data by using schema-aware retrieval, strict SELECT-only validation, and local LLM inference. Tests showed that it was very correct, could stand up to common security attacks, and worked the same way on both local and remote databases. The experiments that compared models showed that bigger LLMs are more accurate, while smaller models speed up response time. The test against a commercial solution showed that QueryMind is just as accurate as the competition while keeping privacy and full local control. These results show that a stable and reliable prototype can be made with RAG-guided context, clear validation rules, and a modular design.

The project also showed some problems that will affect what happens next. The main problem is the time it takes to generate LLM, and the system is only meant to support read-only operations. The prototype only works with one SQL dialect and needs good schema metadata to be accurate. The system does not support multi-turn dialogue, and smaller models have trouble with queries that involve more than one table. These limitations don't make the system less valuable; instead, they show where it needs to be improved to be ready for production.

Future work should take QueryMind beyond the limits of its research. Adding safe write operations with confirmation workflows would allow for full audit logging of INSERT, UPDATE, and DELETE actions. Using dialect-specific adapters to support more than one database engine would make things more compatible. Large-scale validation of databases with over a hundred tables would show that they are strong enough for use in businesses. GPU acceleration and model optimization, like CUDA inference and quantized models, would cut the average time it takes to generate something. Multi-turn conversational queries would make things easier to use by letting users narrow down their results with simple follow-up questions. These changes would make QueryMind a faster, more powerful, and more useful system that could be used by more people.

## References

- [1] M. &. Company, "The data-driven enterprise of 2025," 28 Jan 2022. [Online]. Available: <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-data-driven-enterprise-of-2025#/>.
- [2] J. Chaurasia, "The State Of Data Quality," 24 Apr 2024. [Online]. Available: <https://www.forrester.com/report/the-state-of-data-quality-2023/RES180814>.
- [3] D. Beerbohm, "Text-to-SQL is Dead – Long Live Text-to-SQL," 23 Oct 2025. [Online]. Available: <https://www.exasol.com/blog/text-to-sql-governance/>.
- [4] C. X. Y. H. Z. Y. Bohan Zhai, "Optimizing Reasoning for Text-to-SQL with Execution Feedback," *aclanthology.org*, 2025.
- [5] A. Chen, "Text-to-SQL for Enterprise Data Analytics," Jul, 18, 2025.
- [6] M. Kapadia, "Text2SQL.ai," [Online]. Available: <https://www.text2sql.ai/>.
- [7] I. Sommerville, "Software Engineering," in *10th ed.*, Pearson, 2016.
- [8] R. Pressman, "Software Engineering: A Practitioner's Approach 8th ed.," McGraw-Hill, 2015.
- [9] "MySQL Sample Database," MySQLTutorial, [Online]. Available: <https://www.mysqltutorial.org/getting-started-with-mysql/mysql-sample-database/>.
- [10] J. C. R. H. F. N. R. H. Fernando Vedoin Garcia, "Managing Risks in Agile Methods: a Systematic," 2021.
- [11] Ollama, "Models," <https://ollama.com/search>, [Online]. Available: <https://ollama.com/search>.
- [12] FreeSQLdatabase. [Online]. Available: [www.freesqldatabase.com](http://www.freesqldatabase.com).

# Appendix A

## Appendix A: Requirement Elicitation Interview Questions

1. What types of users do you expect to interact with a Text-to-SQL system in your environment?
2. How important is it that the system restricts SQL operations to read-only queries?
3. What security risks concern you most when using natural-language interfaces for database access?
4. What level of visibility should users have regarding tables, columns, and schema details?
5. How should the system respond when the model generates incorrect or unsafe SQL?
6. What information must be included in error messages to be helpful without exposing sensitive details?
7. How fast should the system respond for it to be acceptable in your workflow?
8. How important is showing the generated SQL to users before execution?
9. What safeguards should exist against SQL injections or malformed queries?
10. What criteria define a “trusted” or “acceptable” output from the system?
11. Are there scenarios where the system should block user queries entirely? If yes, based on what conditions?

## Project Website & Resources



*Scan the QR code or click it to access our website*